Theory of Computation.

It is the branch of computer science that deals with how efficiently a problem can be solved on a model of computation using an algorithm.

TOC
- Automata Theory & languages
- Computability Theory
- Complexity Theory

1) Automata Theory & lang :-
Deals with definition & properties of various mathematical model of computation.
* Study of machines & problems which can be solved by using these machines.
↓
eg Finite Automata, Turing Machine.

2) Computability Theory :- It deals with what can be, what can't be computed by the model.

3) Complexity Theory :- It groups the computable problems based on their hardness.

→ Purpose of T.O.C :-
Develop formal mathematical model of computation that reflect real world computers.

Date: / /
Page No.

Theory of Computation:

Theory of Computation is the branch of Theoretical Computer Science & Mathematics that deals with what problems can be solved on a model of computation using an algorithm & how efficiently they can be solved or to what degree.

eg approximate solns vs precise ones.

Automata: Came from Greek word (Auto/Mata) which means something is doing something by itself.

Theory of Computation: → what problem can be solved
→ which Algo. can be used
→ How efficiently

↓
mnz to execute something
i.e we give i/p & it will produce o/p.

Machine
(Automata).

eg If we have any problem, to solve that problem we can write an algorithm for that problem. or if we can design a machine for that problem then it will be called as computable.
But if that problem is not solvable by Computer, or if we can't design can machine for that problem it is called non-computable then we design a machine Called 'Automata'.

eg If we have problem P_1 → Accept all C-valid programs.

→ Can we design a machine corresponding to problem P_1 → Yes bcz we have a s/w called compiler which will check all C-pgms.

Now if we have problem P_2.
Accept all C.valid pgms which never goes into infinite loop. → NO
It will not be computable.
So for these Non-computable problems we design machines eg F.A, PDA, LBA, Turing Machine etc

eg If we have a language which consists of strings start with 'a'.

Fundamental Terms:-

1. Symbol: Symbols are the basic building block which can be any character.
eg a,b,c,---z. letters.
0,1,2,---9. Numbers / digits.
+, -, %, *, — special characters.

2. Alphabet: An alphabet is a finite non-empty set of symbols. In T.O.C we use 'Σ' to designate alphabet. Alphabet is denoted by Sigma 'Σ'.
eg $\Sigma = \{a, b, \ldots z\}$ Set of all lowercase letters.
$\Sigma = \{a, e, i, o, u\}$ set of vowels.
$\Sigma = \{+, \&, \%, \ldots\}$ set of all special characters.
$\Sigma = \{0, 1, 2, \ldots\}$ set of whole nos.

3. String / word: Finite set of sequence of symbols choosen from some alphabets.
eg. $w_1 = a, b, c, d$ choosen from lowercase alphabet $\Sigma = \{a, b, c, \ldots\}$
w_1 is a string.
eg $w_2 = 01001$ is a string from binary alphabet $\Sigma = \{0, 1\}$
eg $w_3 = abaacb$ is a string from alphabet $\Sigma = \{a, b, c\}$

NULL string/
Empty string: Empty string is the string with zero occurances of symbol (no symbol in set). It is denoted by 'ϵ' or { }.

Length of string: It is the no. of symbols in the string/word. It is denoted by |w|.

eg. w = 01011O your string alphabet Σ = {0,1}
length of string |w| = 6.

Concatenation of string: Joining 2 or more strings. It is defined as the string formed by making a copy of string x followed by a copy of string y, if we have 2 strings x & y.
→ (It's not commutative
eg $w_1 \cdot w_2 \neq w_2 \cdot w_1$)

eg $w_1 = a\,b$ $w_2 = b\,a$
$w_1 w_2 = abba$
$w_2 w_1 = baab$

Let $x = a_1 a_2 a_3 \ldots a_n$ &
$y = b_1 b_2 b_3 \ldots b_n$
Concatenation of string $xy = a_1 a_2 a_3 \ldots a_n b_1 b_2 \ldots b_n$
eg S = ababa & t = cdcddc
Concatenation of St = ababacdcddc

Substring:
Power of an alphabet
Any string of consecutive symbols in some string 'w' can be collectively said as a substring.
eg w = abab
is substring = ab, a, ba etc

$\frac{n(n+1)}{2} + 1$ = No. of Substrings

Date : / /
Page No.

eg SUBSTRING

(1) PUTG (X). Not consecutively
(2) RGS X → Not Consecutive
(3) STR ✓ → Consecutive
(4) UBS ✓
(5) ε → Epsilon (includes in every string).

Q. Find total no. of substrings in

NIGHT

Power of an Alphabet (Σ). 2^n = length

$\Sigma = \{a, b\}$.

Σ^0 = Set of all strings with length 0. = $\in$

Σ^1 = Set of all strings with length 1. $\{a, b\}$

Σ^2 = Set of all strings with length 2 $\{aa, ab, ba, bb\}$

Σ^3 = Set of all strings with length 3.
$\{aaa, aab, aba, abb, baa, bab, bba, bbb\}$

Σ^* → (Kleene closure). (infinite strings)

Σ^* = Set of all combinations including NULL on Σ including $\in$ (infinite language).
Set of all strings

eg $(ab)^* = \{ab, \phi, abab, ababab, \dots\}$. $a^*b = \{ab, b, aab, aaab, \dots\}$.

Σ^* = If Σ is a set of symbols then we use Σ^* to denote the set of strings obtained by concatenating zero or more symbols from Σ of any length in generating any string of any length which can have only symbols specified in Σ.

$\Sigma^* = \{a, b\}$

$\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots \cup \Sigma^\infty = \Sigma^*$

Date : / /
Page No.

Positive closure: Σ^+.

Excluding ε in Σ^* is called Σ^+.

$$\Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cdots \cup \Sigma^\infty = \Sigma^+$$

Set of all possible combinations excluding NULL

eg:

$a^+ = \{ a, aa, aaa \}$.

$(ab)^+ = \{ ab, abab, ababab, \cdots \}$.

$ab^+ = \{ a, ab, abb, abbb, abbbb \cdots \}$.

Language:

Collection of all strings.

Language is defined as a set of strings.

If Σ is an alphabet and $L \subseteq \Sigma^*$, then L is a language.

eg If we have $\Sigma(a, b)$.

L_1 = strings of length 3

$= \{ aaa, aab, aba, abb, baa, bab, bba, bbb \}$.

↓ Finite language

Finite lang

L_2 = Set of all strings of length 2

$= \{ aa, ab, bb, ba \}$

L_2 = Set of all strings of length 3

$= \{ aaa, aab, aba, abb, baa, bab, bba, bbb \}$

Infinite lang

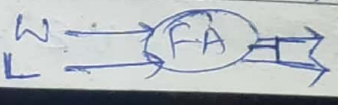
L_3 = Set of all strings that start with 'a' and end with 'a'.

$= \{ a, aa, aaa, aaaa, \cdots \}$.

Finite Automata:-

$w \in L$ or $w \notin L$

In order to find whether given string belongs to a language or not is decided by F.A



language formed over Σ (alphabet) can be Finite or Infinite

Date : / /
Page No.

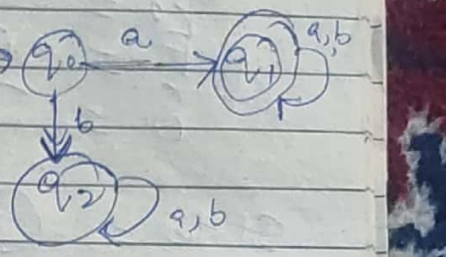
Language: Collection of strings bounded by some condition, formed from Σ.

Grammar — Language — Automata

$\Sigma = \{a, b\}$ — Generate — accept

$S \to aX$
$X \to aX \mid bX$

$L = \{a, aa, ab\}$



Automata represents a language

Finite Automata :-

→ It is basically finite representation of language (which can be infinite) which can be used to decide whether a string belongs to a language or not.

→ It is a model (machine) which recognizes set of valid strings of a particular language

→ It takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then transition occurs.

→ At the time of transition, the automata can either move to the next state or stay in the same state

→ Finite Automata has two states, Accept state or Reject State, when the string is processed successfully & the automata reached its final state, then it will accept

Formal Definition of Automata:

Date: / /
Page No.

A finite Automata is ~~simplest model~~ of computation which consists of 5 components or tuple. $(Q, \Sigma, \delta, q_0, F)$

Q = Set of states.
Σ = Finite set of input symbols.
q_0 = Initial state
F = Final state
δ = Transition function.

$$\delta : Q_1 \times \Sigma \to Q_0.$$

States = ○
Initial state = →○ or (−)
Final state = ◎ or (+) or (*)

Finite Automata

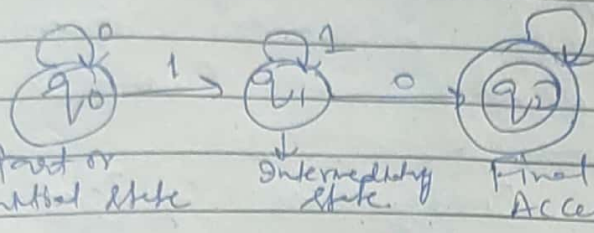
- F.A with output
  - Moore Machine
  - Mealy Machine
- F.A without output
  - DFA
  - NFA
  - ∈-NFA

(Q, Σ, F, q_0 → same, differ in δ)

F.A is an abstract computing device. It is a mathematical model of a system with discrete inputs, outputs, states and set of transitions from state to state that occurs on input symbols from alphabet Σ.

It can be represented by 3 ways
Graphical (Transition Diagram)
Tabular (Transition Table).
Mathematical (Transition function or Mapping fun.)

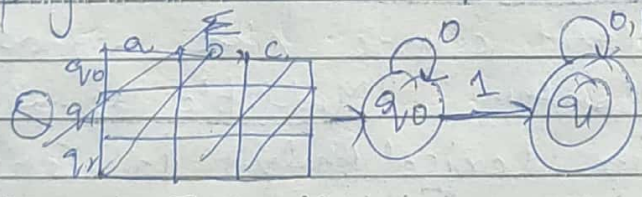
Transition Diagram: (Transition Graph)
It is a directed graph associated with vertices of graph corresponds to the state of finite automata.

eg 

{0,1} are i/ps

Transition table:
It is basically a tabular representation of transition fxn that takes two arguments (a state & a symbol) and returns a value (the next state).

→ Rows correspond to states.
→ Columns " " i/p symbols.
→ Entries " " Next States.
→ start state is marked with an arrow(→)
→ Accepting states are marked with star(*).



	0	1
→ q0	q0	q1
*q1	q1	q1

Transition Function:
→ Transition Fxn or Mapping fxn is denoted by δ.
→ Two parameters are passed to this transition fxn.
i) Current State &
ii) Input Symbol.
→ Transition fxn returns a state which can be called as Next State

δ(Current state, Current i/p symbol) = Next State

$Q_1 \times \Sigma \rightarrow Q_2$

eg $\delta(q_0, a) = q_1$ or $\delta(q_0, 1) = q_1$.

Date: / /
Page No.

Applications of F.A

1. It plays an important role in Compiler design.
2. In Switching Theory & design and Analysis of digital circuits automata theory is applied.
3. Design & Analysis of Complex S/w & H/w systems.
4. To prove correctness of pgm automata Theory is used.
5. To design Moore & Mealy Machine.
6. It is base for formal languages & these formal languages are base for Programing languages.

Deterministic Finite Automata (DFA)

→ In DFA, On every state every symbol is defined
→ In DFA, one symbol is read at a time
→ There is only one path for specific i/p from current state to next state
→ DFA does not accept Null move i.e the DFA can't change state without any i/p.
• DFA can Contain Multiple final states.
• It is used in Lexical Analysis in Compiler

Non-Deterministic Finite Automata (NDFA) (NFA)

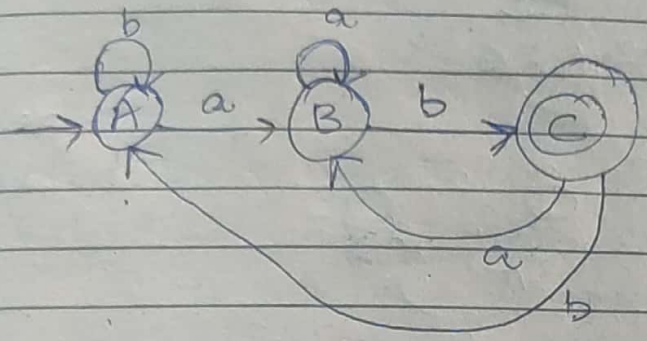
→ In NFA, there exists many paths for specific i/p from current state to next state
→ It Contains multiple next states & it Contains ε transition.
• Every NFA is not DFA, but each NFA can be Converted into DFA
• ε move is allowed.
• Designing is easy.

$|w| = 2$ ——— exactly 2 = (n+2) states.
$|w| \leq 2$ ——— at most 2 = (n+2) states.
$|w| \geq 2$ ——— at least 2 = (n+1) states.

Date : / /
Page No.

Q Construct a DFA which accepts set of all strings over $\Sigma = \{a, b\}$ which ends with 'ab'.

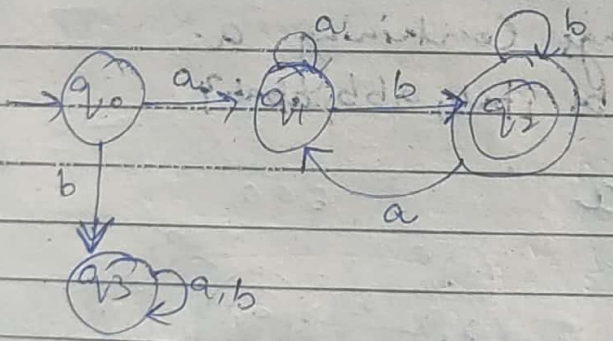
$L = \{ab, aab, bab, abab, ababab, \ldots\}$



abab ✓
abbab ✓
abbabbabab ✓

Q. Set of all strings over $\Sigma = \{a, b\}$ starting with 'a' & ending with 'b'.

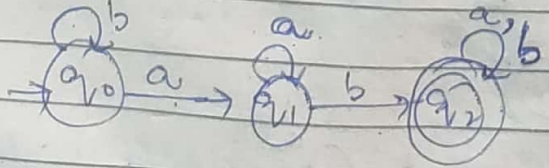
$L = \{ab, aab, abb, aabb, abab, \ldots\}$



aababb

Q. Set of all strings over $\Sigma = \{a, b\}$ which contain 'ab' as a substring.

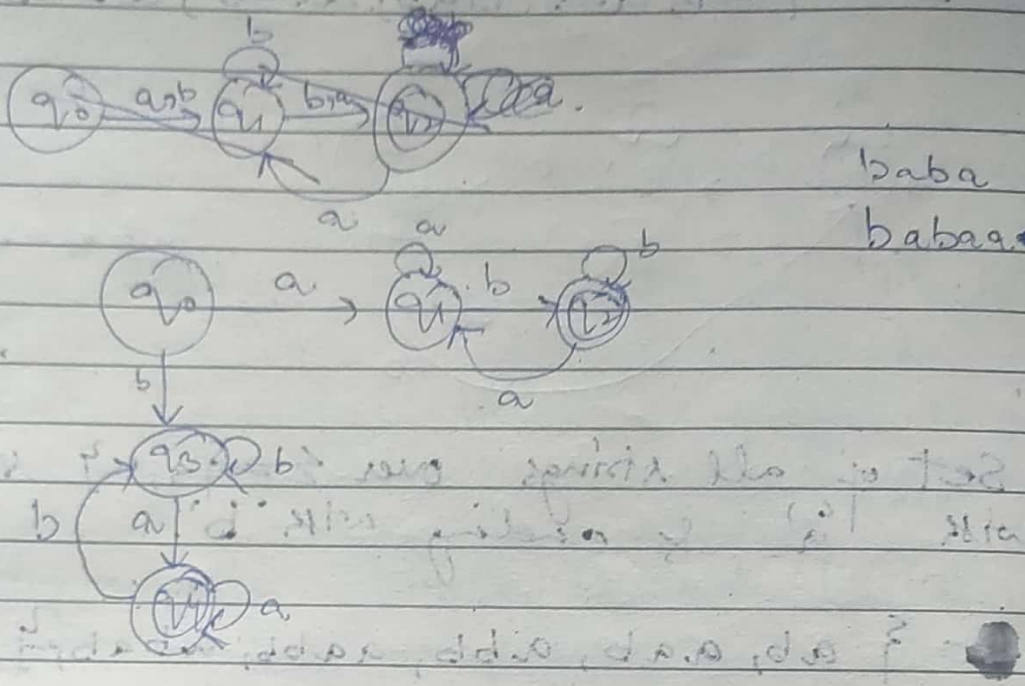
$L = \{ab, aab, bab, aabb, baba, \ldots\}$.



Date : / /
Page No.

Q Set of all strings over $\Sigma = \{a, b\}$ starting & ending with diff symbol.

$L = \{ab, ba, abb, baa, a^*bab, aaab \ldots\}$

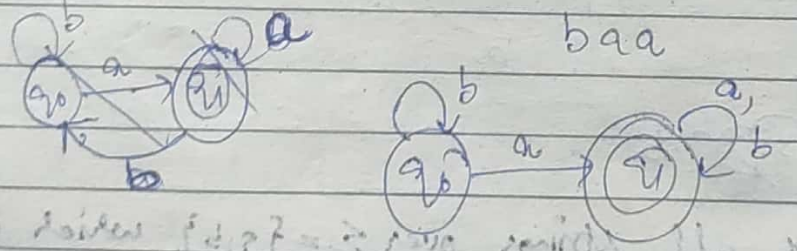


baba

babaa

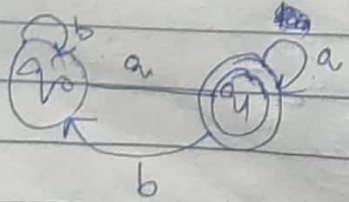
Q. Accept all strings containing a

$L = \{a, ab, aba, ba, bab, abb, abab\}$



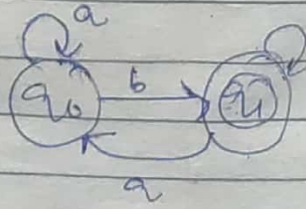
Q. End with 'a'

$L = \{a, ba, baa, aa, aba, bba, \ldots\}$



Q. Ends with 'b'.

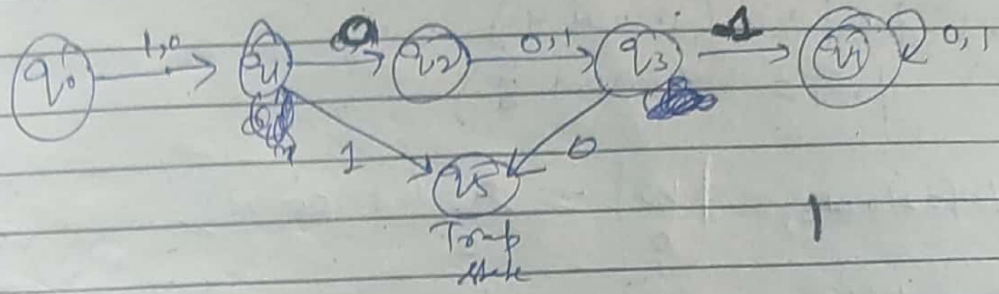
$L = \{b, ab, bb, abb, aab, bab, \ldots\}$



$\Sigma = \{0,1\}$.

Q. Second symbol is 0 & 4th symbol 1.

$L = \{1001, 0001, 1011, 0011, \ldots\}$. _0_1



Q. Accept all strings over $\Sigma(0,1)$ divisible by 3.

0 1 2 = Remainder.

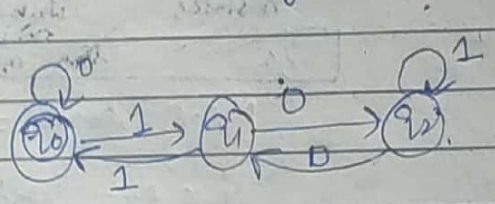
All nos divisible by 3 have remainder 0, 1 or 2.

"101" = 5/3 = 2

"1001" = 4/3 = 1

"011" = 3/3 = 0

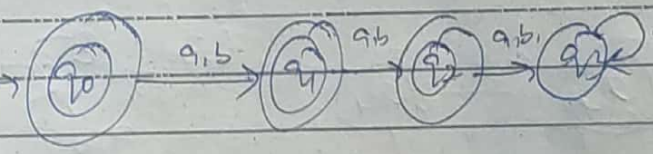
"001" = 1/3 = 1



F.A which accepts binary nos divisible by (n), no. of states = n

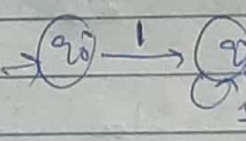
Q. $|w| \leq 2$ (atmost 2). $\Sigma = \{a,b\}$.

$L = \{\in, a, b, aa, bb, \ldots\}$



$\Sigma = \{0,1\}$.

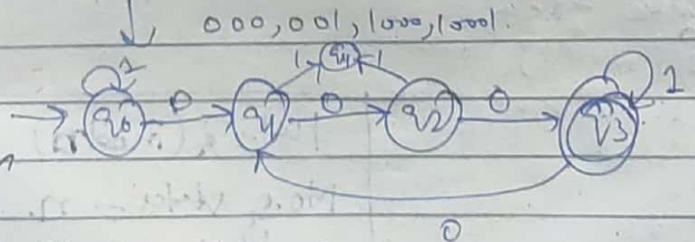
Q. Starts with 1 & ends with 0



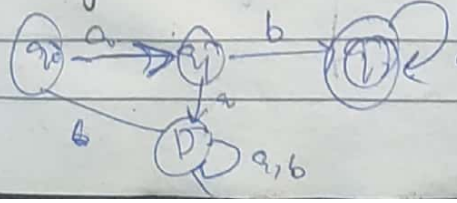
Q. accepts only i/p 101.

Q. with 3 consec. 0's.

000, 001, 1000, 1000



Q. Starts with ab.



Min. no. of states required = n+2

1 [illegible] 05

Date : / /
Page No.

Design DFA over $\Sigma = \{a,b\}$ such that every string accepted must

a) $|w| = 2 \pmod 3$

b) $|w| = 3 \pmod 4$

c) $|w| = 1 \pmod 5$

a) ~~$|w| = 2 \pmod 3$~~ $|w| = r \pmod n$

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 ... n-1

→ From here lengths we have to find those which when divided by 3 gives remainder 2 = 2, 5, 8, 11.

→ Here the possibility of remainder = 0, 1, 2 when we divide any no. by n then remainder range will be from 0 to n-1.

(when any no. will be divided by 3 then remainder will be 0 to n-1 i.e. 0, 1, 2 only)

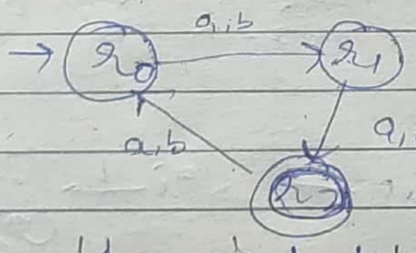
r mod n	
~~If n is odd → n states~~	~~If n is even. If we can write even no. in 2^m form then m+1 states~~ ~~If we can't write 2^m form then $2^x \cdot p$ → x+p states~~

No. of states = n.

i.e. No. of states will be which is given in mod

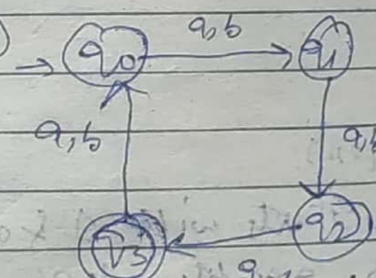
eg. 2 mod (3) → No. of states = 3

(a)

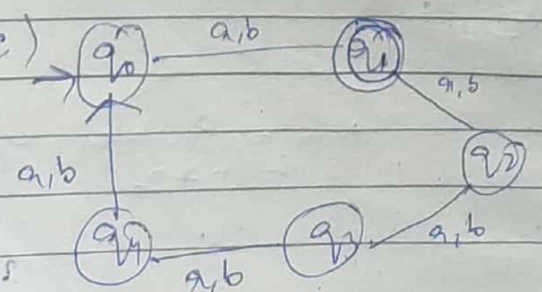


Here final state will be q_2 bcoz we have given 2 (mod 3), i.e. it will be final state

(b)



(c)



$|w| = r \pmod n$

No. of states = n.

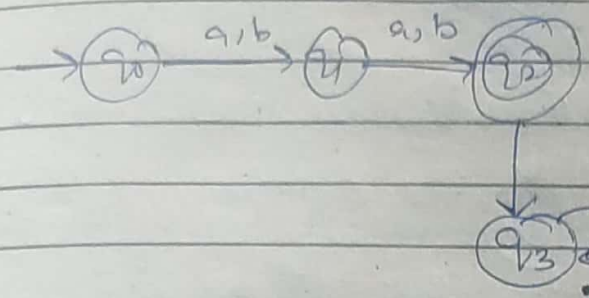
final state = r.

Q Design DFA for $\Sigma = \{a, b\}$.

$|w| = 2$ $|w| \geq 2$ $|w| \leq 2$.

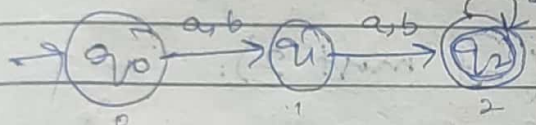
(a) $|w| = 2$

$L = \{aa, bb, ab, ba\}$.



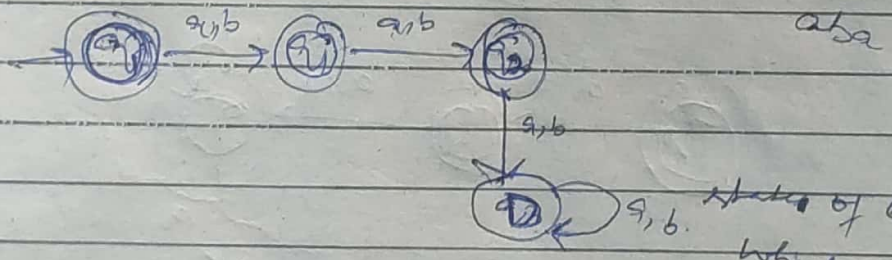
(b) $|w| \geq 2$ (i.e at least length should be 2).

$L = \{aa, bb, ab, ba, aaa, aba, abb, bab, baa, \ldots\}$



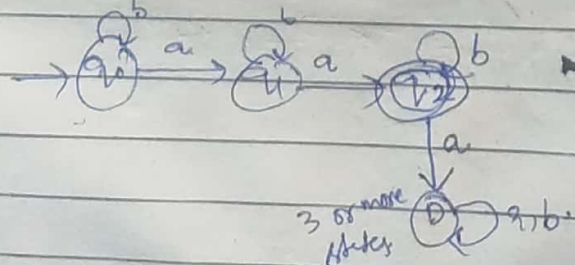
(c) $|w| \leq 2$ (i.e at most length should be 2)

$L = \{\epsilon, a, b, aa, bb, ab, ba\}$



Q Construct DFA which accepts set of all strings over $\{a, b\}$ such that $n_a|w| = 2$ (length of no. of "a's" should be 2).

$L = \{aa, baa, aba, aab, bbaa, \ldots\}$.

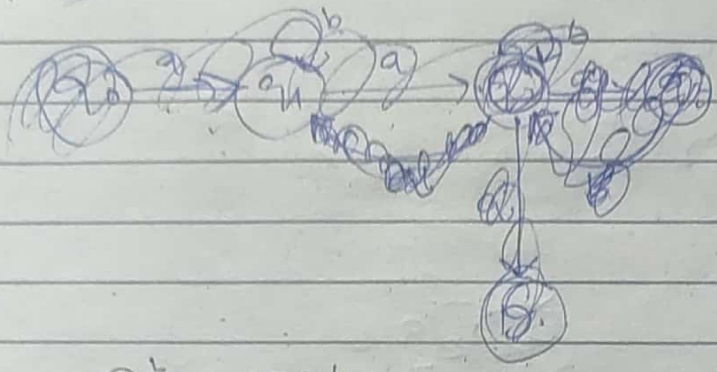


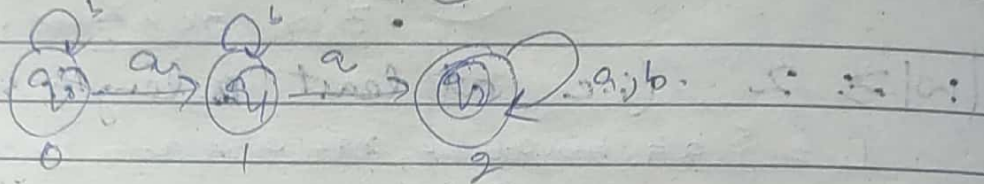
Date : / /
Page No.

Q Construct DFA for:

$n_a(w) \geq 2$. (at least 2)

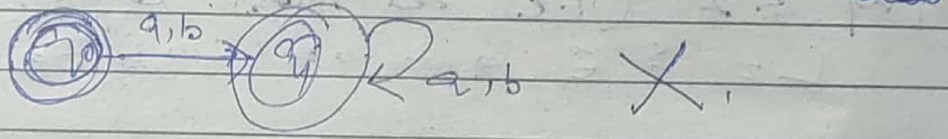
$L = \{aa, aaab, aab, aabb, aabbb, aba, aabaa, \ldots\}$ baa

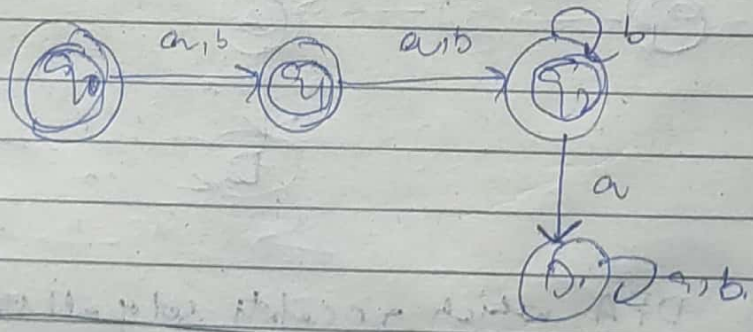




Q $n_a(w) \leq 2$. (at most 2).

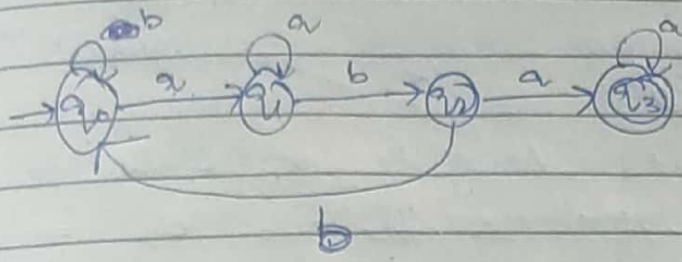
$L = \{\in, a, a, b, ab, abb, aab, \ldots\}$





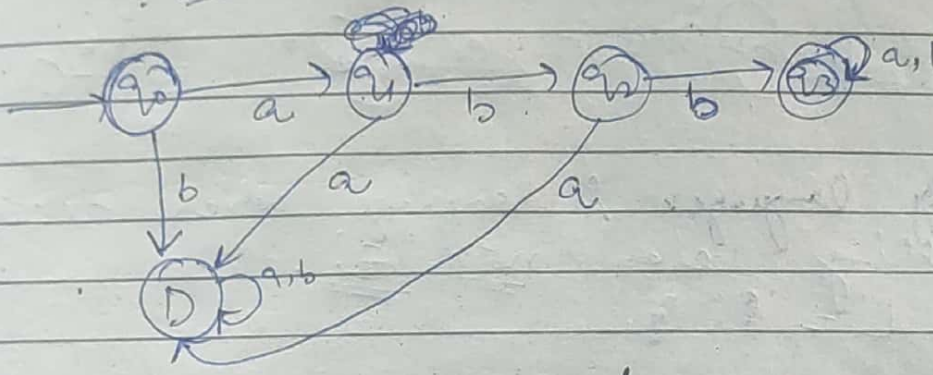
Date : / /
Page No.

Q Design a DFA, which contains substring s= "aba"
L = { aba, aaba, baba, babab, abaa, ababa, ... }

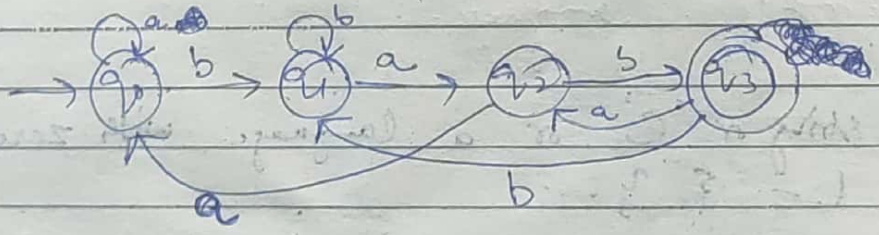


abbaba

Q Design DFA, which starts with substring S = "abb"
L = { abb, abba, abbb, abbbb, abbab, ... }

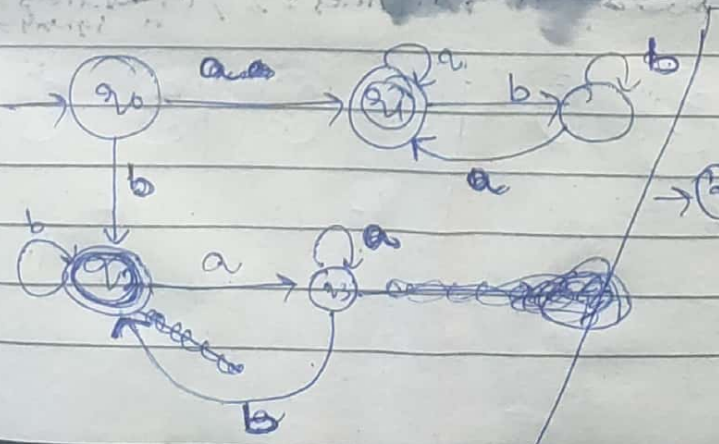


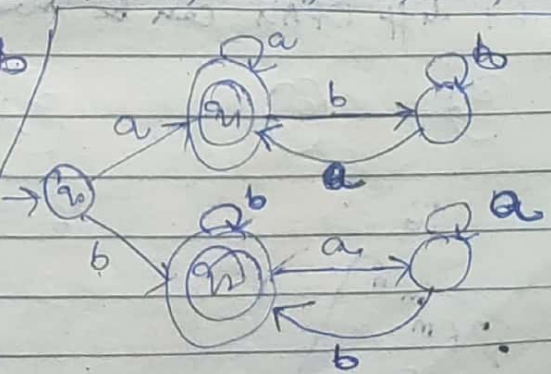
Q Design DFA, which ends with 'bab'.
L = { bab, abab, bbab, abbab, babab, aabab, bbbab, ... }



babbab
bbbab
babbab

Q. Design DFA which starts & end with same Symbol.
L = { a, b, aa, bb, aba, bab, abba, bbbb, aaaa, ababa, babb, babab, ... }

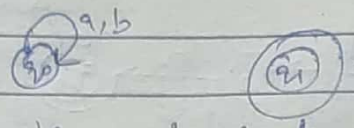




{ 20 diff DFA's will be there which will accept Universal language.
20 diff DFA's will be there which will accept nothing.

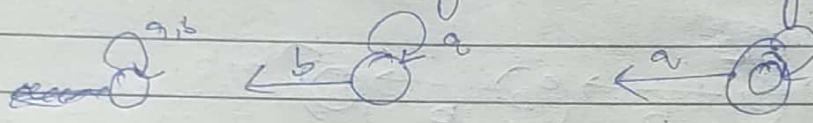
Date : / /
Page No.

Unreachable state : The state to where from initial state we will never reach.

eg 

→ So we can't reach to q_1 ever as 'q_0' has made transitions over (a,b) on 'q_0'. So we can't reach to q_1 ever.

→ q_1 can go to q_0 by 4 ways.



Empty language : A language which accept nothing even not 'ϵ'.

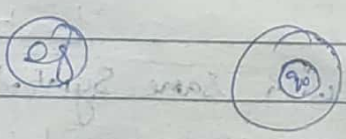
$L = \{\ \}$.

eg 

Here these two states will be representing a empty language

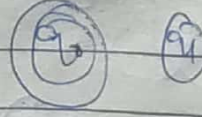
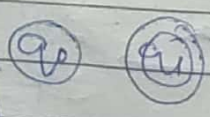
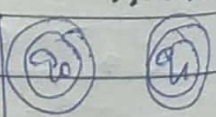
Null string or 'ϵ' or a language with zero string.

$L = \{\epsilon\}$.

eg 

→ It represents states for ϵ or Null string.

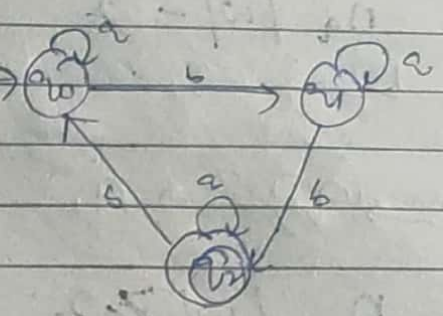
* 64 diff DFA's can be made for no. of states = 2, Transitions = 2 or symbols = 2

			
16	16	16	16.

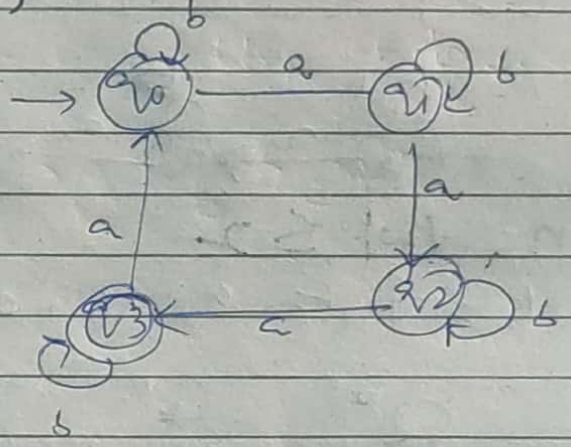
m^n
$n \times m$
$2^2 \times 2^2 = 2^{4+2} = 2^6$

Q. Design DFA over $\Sigma = \{a, b\}$ such that every string accepted must:

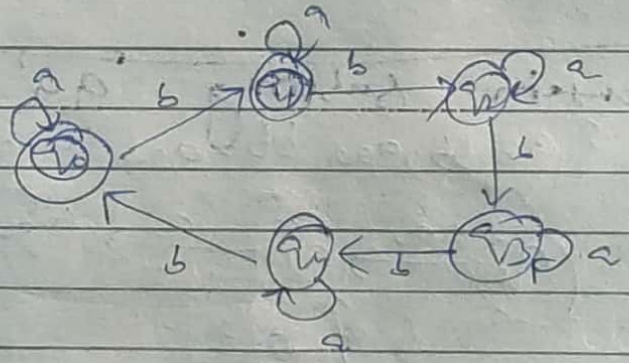
(a) $|w|_b = 2 \pmod 3$



(b) $|w|_a = 3 \pmod 4$



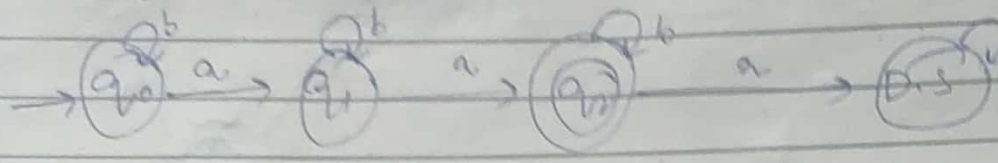
(c) $|w|_b = 1 \pmod 5$



Q. Design DFA which accepts set of strings over $\Sigma = \{a, b\}$

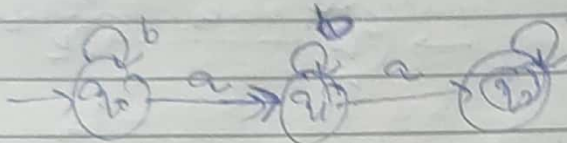
1. $n_a(w) = 2$

$L = \{aa, aab, baa, abba, bbaa, baab, abab, baba\}$



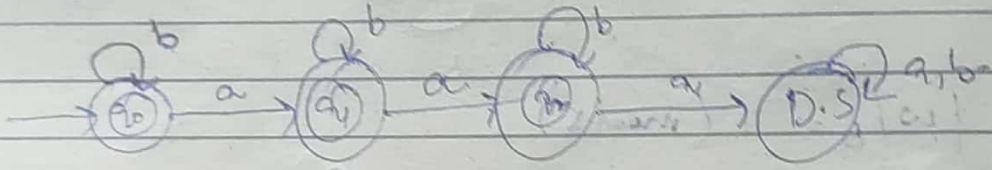
2. $n_a(w) \geq 2$

$L = \{aa, aaa, aab, baa, bbaa, babaa, baba, \}$



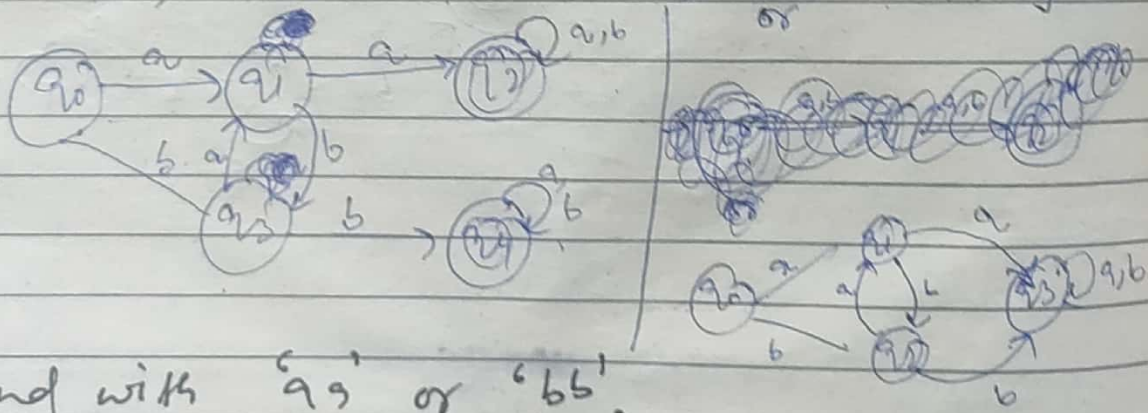
3. $n_a(w) \leq 2$

$L = \{\epsilon, a, aa, ab, b, ba, bab, aba, \}$



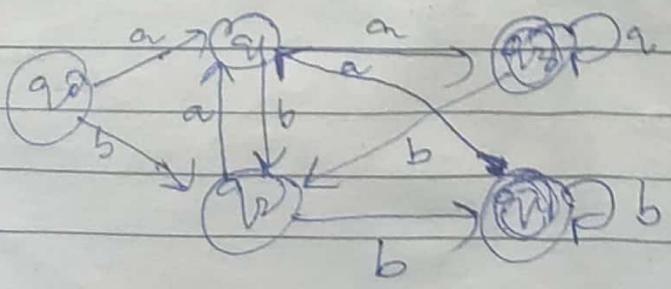
9. Contain substring 'aa' or 'bb'

$L = \{aa, bb, aaa, bbb, aab, bba, aba, babb, abb, baa, \}$



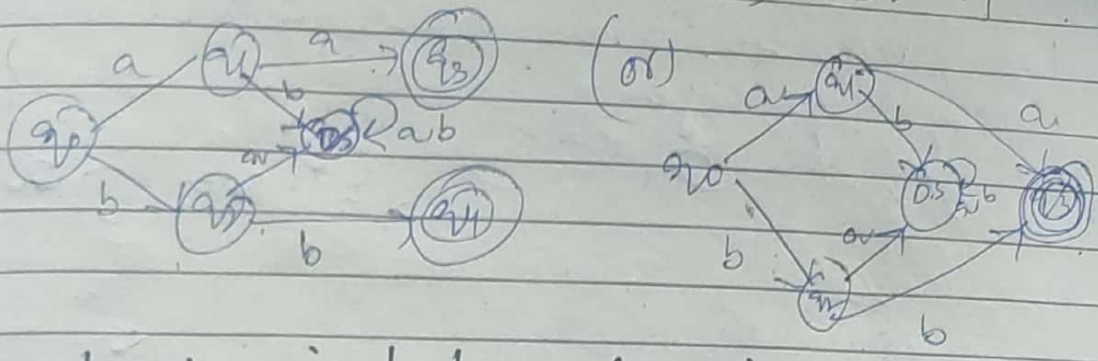
10. End with 'aa' or 'bb'.

$L = \{aa, bb, abb, baa, bbaa, aabb, babb, abaa$

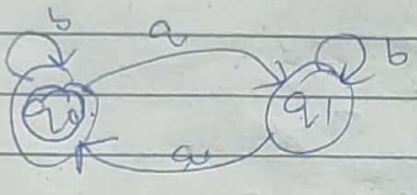


11. Start with aa or bb.

$L = \{aa, bb, aabb, bbaa, aabb, bbaa, bbb, \ldots\}$

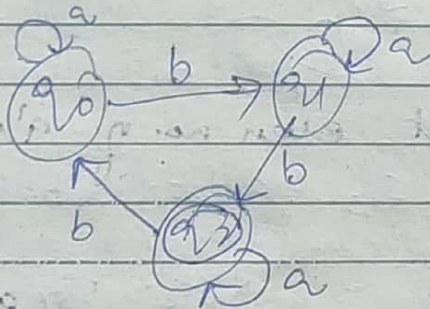


Q. number of a is $|w|_a = 0 \pmod 2$.



States = 2
F. state = 0,

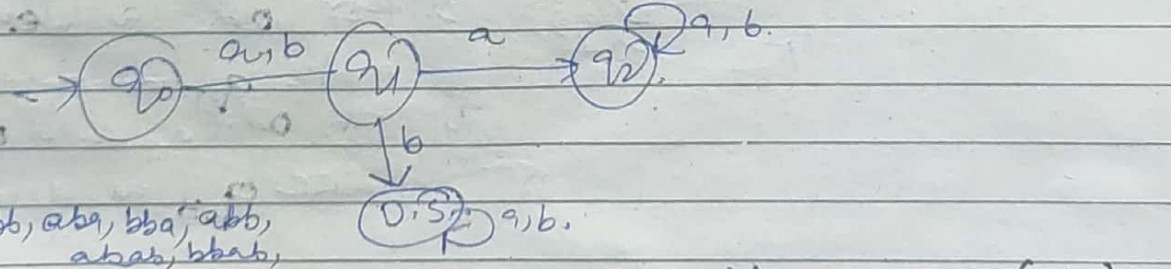
Q. no. of bb is $|w|_{bb} = 2 \pmod 3$



States = 3
F. state = 2

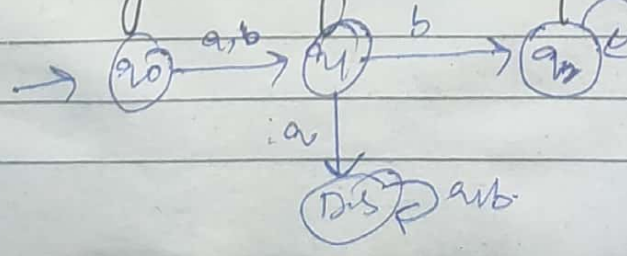
Q. 2nd symbol from left end is always 'a'.

$L = \{aabb, abbb, baba, aa, ba, aab, bab, aba, baa, \ldots\}$



$L = \{ab, bb, aba, bba, abb, abab, bbab, \ldots\}$

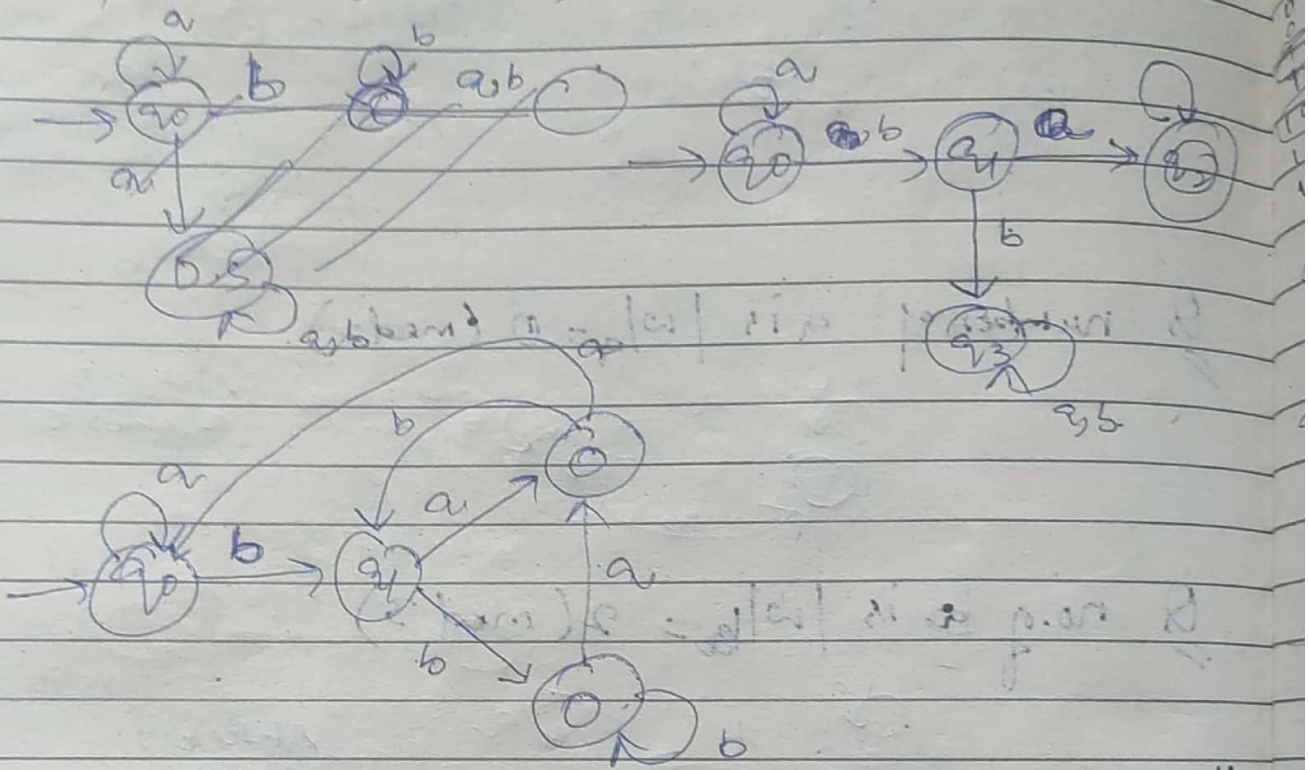
Q. 2nd symbol from left end is always 'b'.



Date : / /
Page No.

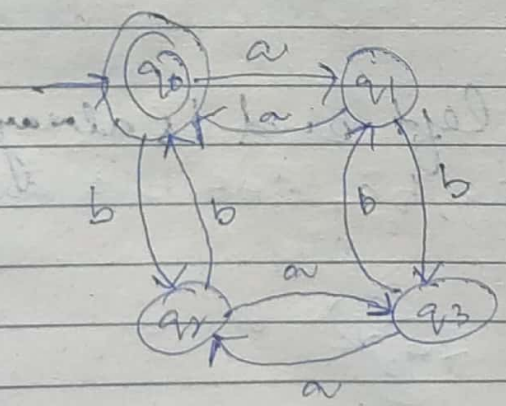
Q. 2nd from right end is always 'b'.

L = { ba, bb, abb, aabb, aabba, ababa, ... }



Q. String accept even no. of a's & even no. of b's.

L = { aabb, ε, aa, bb, abab, baba, baab, abba, baab. }

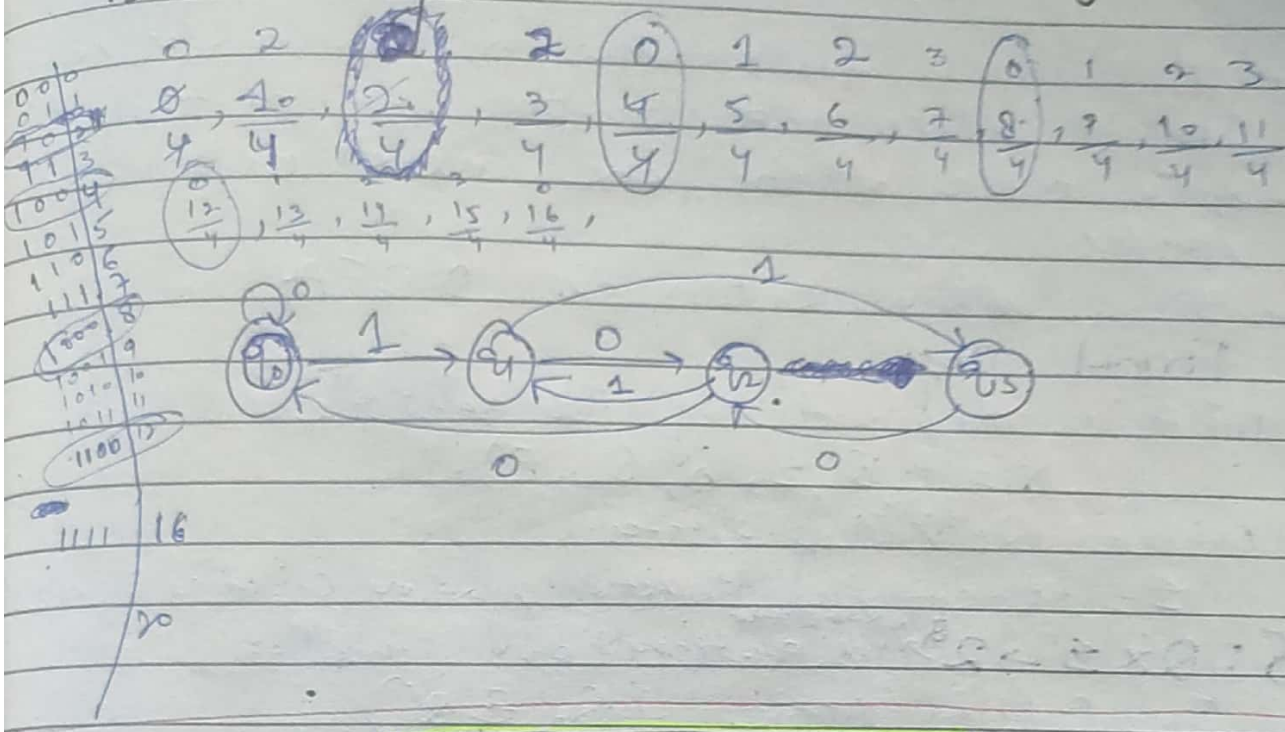


Note:- For even no. of a's & b's.
1. make 'q0' as final state.
2. For even 'a' and odd 'b'.
make q2 as final state.
3. For odd a & even b.
make q1 as final state.
4. For odd a and odd b.
make q3 as final state.

O(mod n).

Date: / /
Page No.

Q Construct DFA for all binary strings divisible by 4.



* All string starting with 'n' length substring will always require minimum (n+2) states in DFA

* All strings ~~starting~~ ending with 'n' length substring will always require minimum (n+1) states in its DFA

Date : / /
Page No.

NDFA / NFA (Non-Deterministic Finite A.)

→ It is an automata in which for some current state and input symbol, there exists more than one next output states.

→ It is also known as NDFA

Formal Defn.

NFA is defined by ~~Quadruple~~ 5 tuples

$M = (Q, \Sigma, \delta, Q_0, F)$.

where Q = finite set of states.

Σ = Non-empty finite set of symbols called as i/p alphabets

$\delta : Q \times \Sigma \to 2^Q$ (no. of states) is a transition function.

Q_0 = Initial state ($Q_0 \in Q$)

F = Final set of states. ($F \subseteq Q$).

→ It is easy to construct an NFA than DFA for a given regular language

→ In NFA, there exist many paths for specific input from current to next state

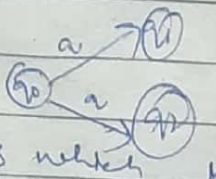
→ Every NFA is not DFA, but each NFA can be translated into DFA

→ NFA can be defined in same way as DFA but with following two exceptions.

- It contains multiple next states.
- It contains ϵ transition.

→ It is not fixed or Deterministic that with a particular i/p where to go next.

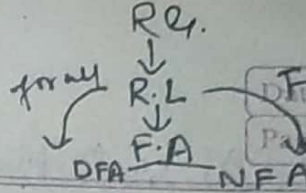
* Non deterministic means if we already know i/p symbol & current state but we can't say what will be the next state eg



* Deterministic, the next state also. Here we know the next state

→ All machines where we are living are of deterministic nature, eg switch.

To make things better: 1. Consistency 2. efficiency



Y we read NFA?

NFA → DFA → MDFA.

→ It is easy to construct NFA theoritically

→ It can't be implemented practically.

power of DFA = NFA.

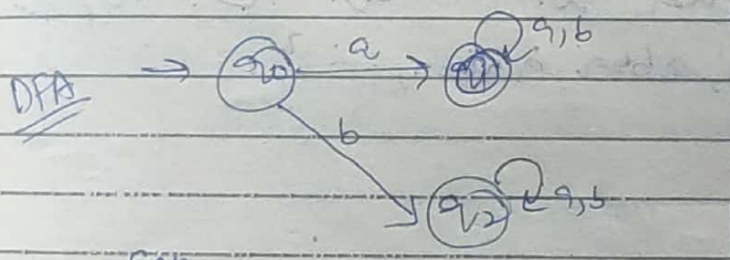
→ So every NFA can be converted into DFA also vice versa, but we don't implement NFA in practical so we don't convert usually DFA to NFA.

Every DFA is NFA

Acceptance by NFA:

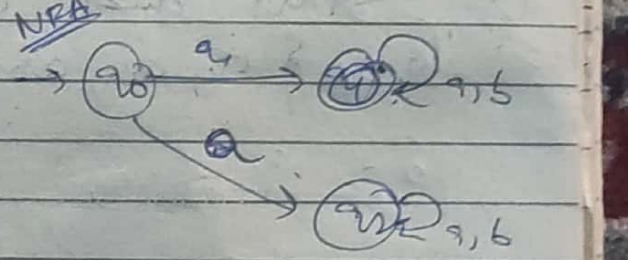
A string 'w' is said to be accepted by NFA if there exists at least one transition path on which we start at initial states & end at final.

$\delta(q_0, w) = F$.



→ Here can we say 'ba' will be accepted here or not

ba
q_0 q_1 q_2 → Non final state, so it will not be accepted.

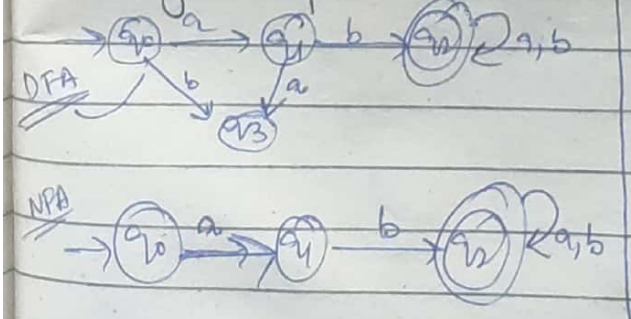


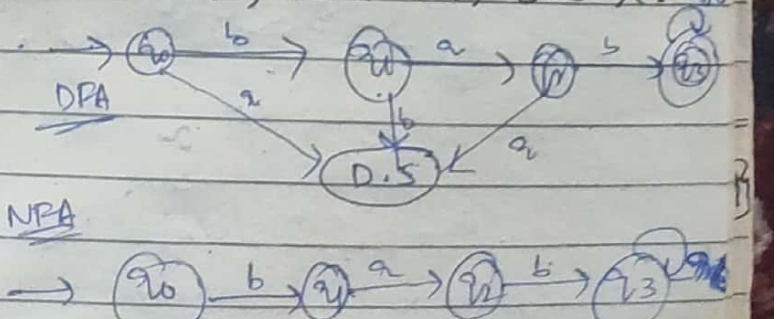
→ Here can we say 'a' will be accepted or not

if it goes to q_1 then accepted

if it goes to q_2 then not.

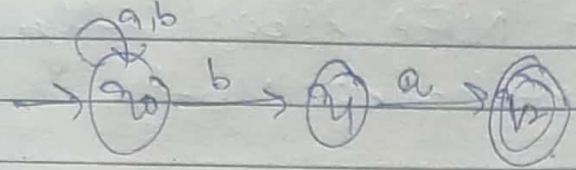
Q Design NFA over alphabet $\Sigma = \{a, b\}$ such that every string accepted must start with (i) abX (ii) bab X.

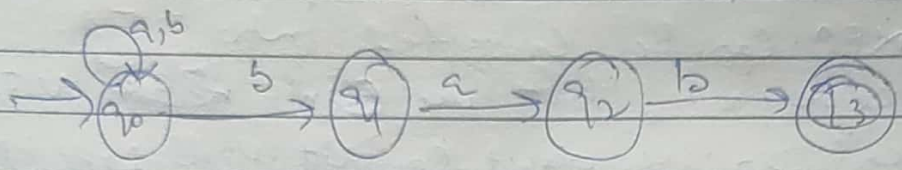




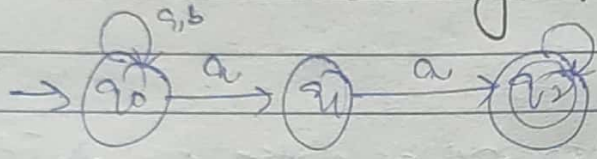
Date : / /
Page No.

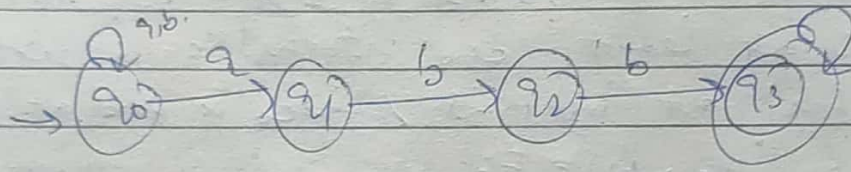
Q Design NFA, such that every string accepted must end with X ba , X bab.



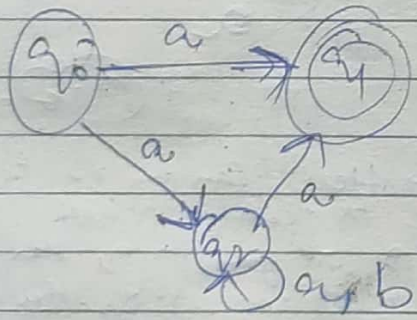


Q Contain Substring. X aa X , X abb X.

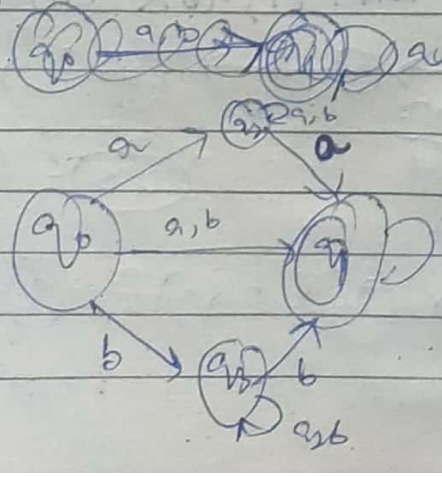




Q Start and end with a. i.e a X a
= L = { a, aa, aba, abba, aabba, ... }.



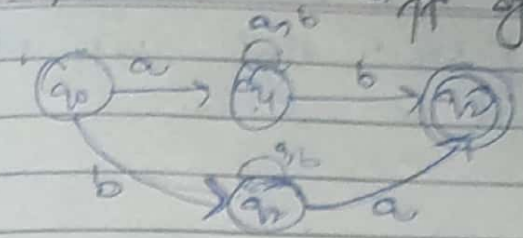
Q Start & end with same symbol. [a, b, aXa, bXb.
L = { a, b, aa, bb, aba, bab, abba, baab, ... }



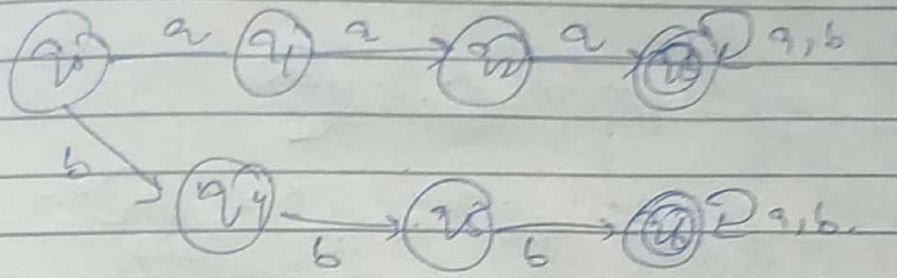
- Start & end with diff Symbols
- Start with Multiple options.
- Ends with Multiple options.
- Contain substring with 'x' options.

Q. Start & end with diff Symbol [a b, ba, aab, bba, abb, ...]

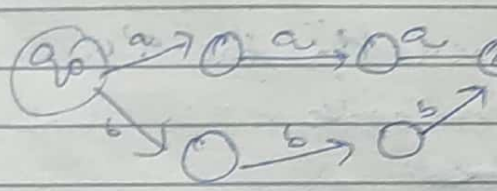
a X b
b X a



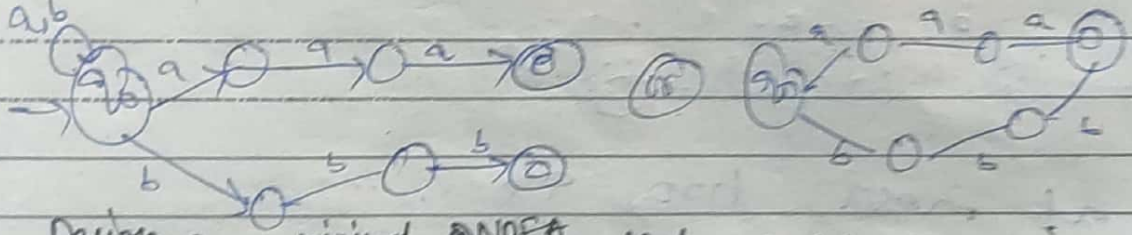
Q. every string w is like $W = SX$, where $S = aaa/bbb$
$L = \{aaa, bbb, aaab, bbba, \ldots\}$



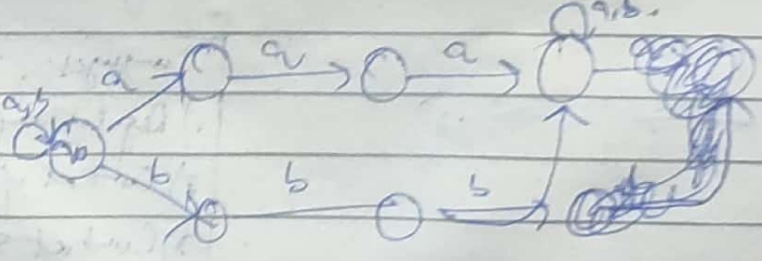
(or)



Q. every string w is like $W = XS$, where $S = aaa/bbb$
$L = \{baaa, abbb, abaaa, baabbb, \ldots\}$



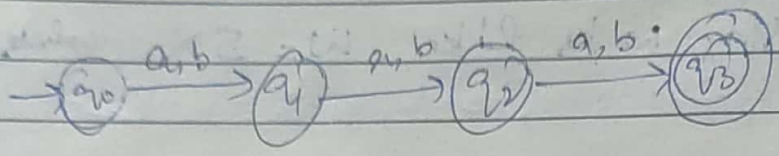
Q. Design a minimal DFA that accepts all strings over alphabet $\Sigma = \{a, b\}$, such that every accepted string w is like $w = XSX$, where $S = aaa / bbb$.



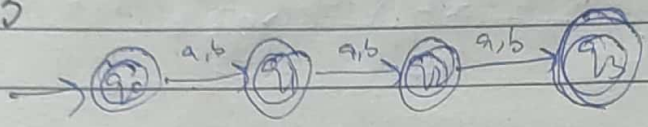
Date : / /
Page No.

Q Design NFA that accepts all strings over alphabet $\Sigma = \{a, b\}$ such that every string "w" accepted must be like

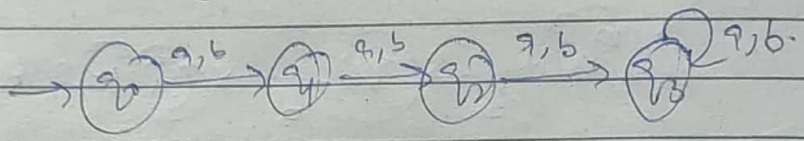
(i) $|w| = 3$.



(ii) $|w| \leq 3$

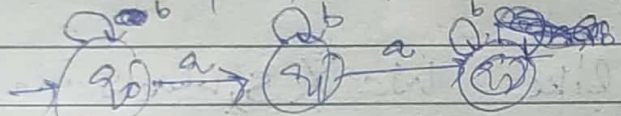


(iii) $|w| \geq 3$

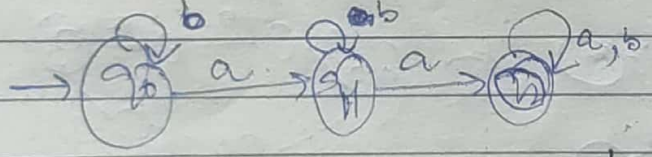


Q string containing exactly two a's.

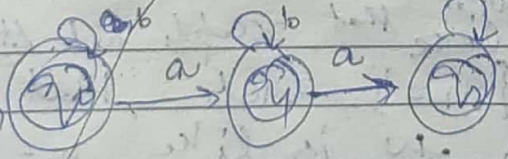
$L = \{aa, aba, bba, bbaa, baab, \dots\}$



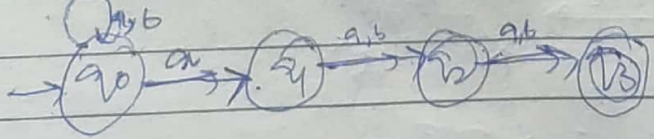
Q at least two a's.



Q at most two a's.



Q 3rd from right end is always a.



Languages.
1) Regular lang.
2) Context free lang.
3) Context sensitive lang.
4) Recursive enumerable lang.

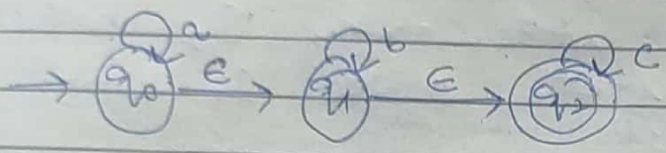
Why we change state?
bcz we don't remember the previous state,
so in order to remember previous state we change state
as in case of FA it don't have its own memory.

Date: / /
Page No.

ε-NFA (or NFA with Null moves)

It is same as NFA, the only difference lies in Transition function.
It has same 5 tupples as in NFA
$(Q, \Sigma, q_0, F, \delta)$.

Here $\boxed{\delta : Q \times \Sigma \cup \{\epsilon\} \to 2^Q}$

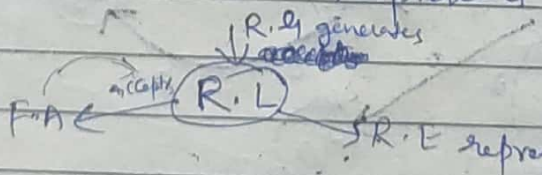


→ If we don't know exactly which state is current state we can use 'ε' there, by which we can stay on both states.

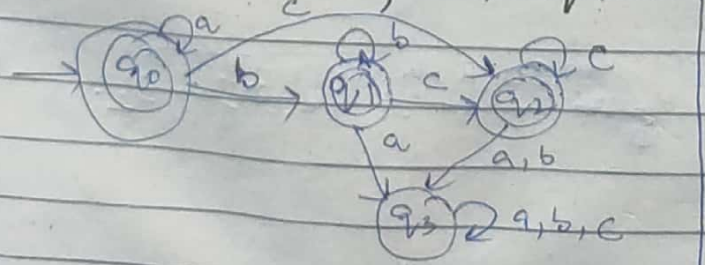
→ In above ε-NFA 'only a' will be also accepted bcz as we are on state q_0 so there is ε transition, from q_0 to q_1 & q_1 to q_2, so we reached q_2.

RE → ε-NFA → NFA → DFA → MDFA

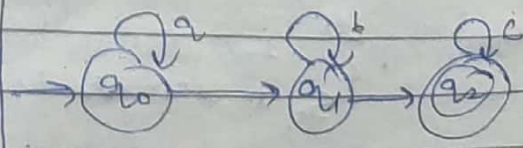
For every lang we can write regular expression, which express F.A. or which represents F.A



Q $L = \{a^n b^m c^q\}$ $n, m, q \geq 0$.



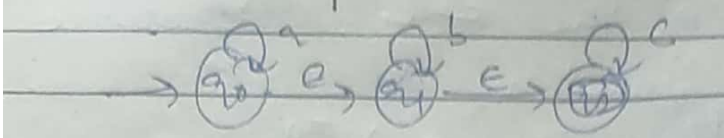
DFA

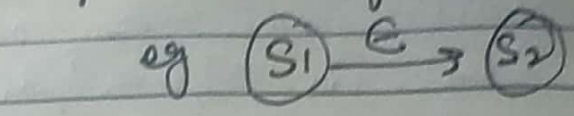


ε-NFA

Date: / /
Page No.

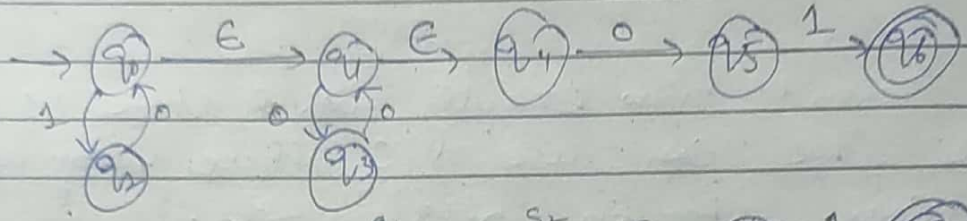
Conversion of ε-NFA to NFA / Eliminating ε-moves



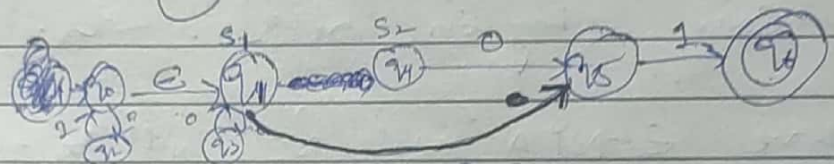
eg 

Algorithm.

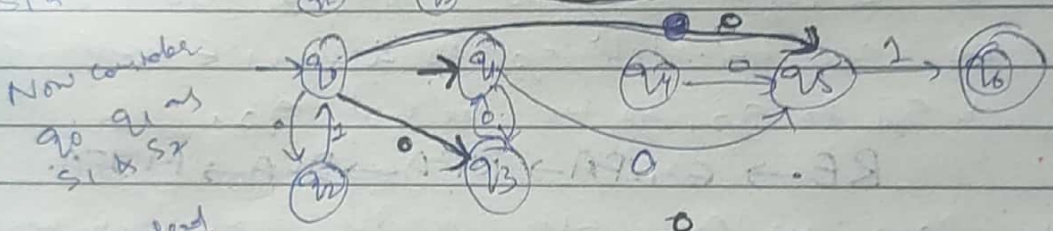
1. Find all edges starting from S_2.
2. Duplicate all edges to S_1 without changing edge labels.
3. If S_1 is initial state, make S_2 initial.
4. If S_2 is final state, make S_1 final.
5. Remove dead state / Unreachable state



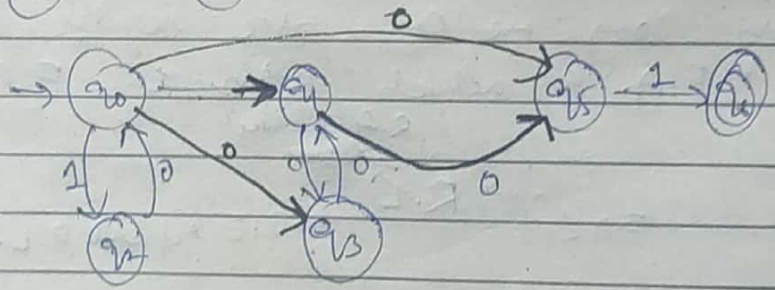
Consider q_1 & q_2 as S_1 & S_2



Now consider q_1 as q_0 S_1 & S_2



Remove dead state

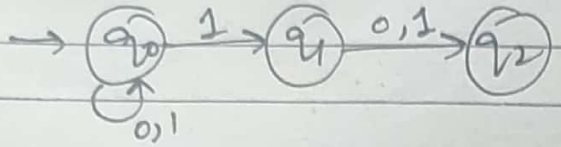


(i) All the edges which are going from S_2, make copy of all those states from S_1 to the states, which it is going from S_2.

Date : / /
Page No.

Conversion of NFA to DFA.

Q NFA of all binary strings in which 2nd last bit is 1



← NFA.

States	0	1
→ q_0	q_0	$q_0 q_1$
q_1	q_2	q_2
(q_2)	—	—

← Transition table of NFA

Now we will make transition table for DFA from above table

States	0	1
→ q_0	q_0	$q_0 q_1$
$q_0 q_1$	$q_0 q_2$	$q_0 q_1 q_2$
$q_0 q_2$	q_0	$q_0 q_1$
$q_0 q_1 q_2$	$q_0 q_2$	$q_0 q_1 q_2$

(Transition table for DFA)

(1) Copy the first row as it is.

(2) Then expand the next term. ($q_0 q_1$).

as states already expanded so no need to expand further

(3) Now $q_0 q_1$ from above table on 0 goes to $q_0 q_2$ & on 1 goes to $q_0 q_1 q_2$

(4) Now expand $q_0 q_2$ as same

(5) expand $q_0 q_1 q_2$ as same

(6) Now all states have been expanded.

(7) Now design DFA for above table.

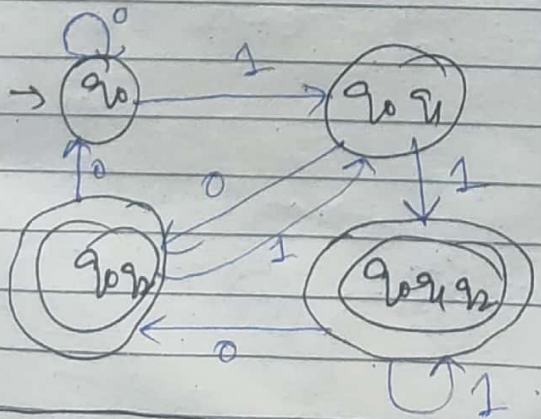
(8) Write all the states.

(9) Make same initial state as in NFA

(10) Make all those states final which contain the final state q_2.

Eg ($q_0 q_1$, $q_0 q_1 q_2$, $q_0 q_2$)

Now design DFA.



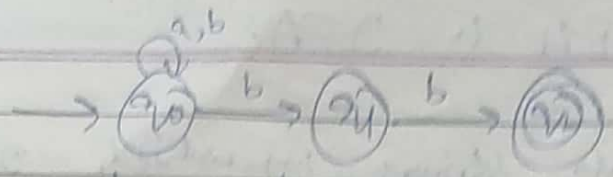
Note: DFA has more states compared to NFA.

If NFA has 'n' states then DFA will be having max. 2^n states

String accepted having bb at least

Date: / /
Page No.

Q.

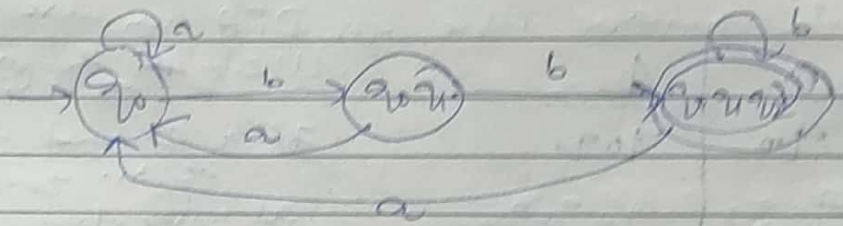


States	a	b
→ q_0	q_0	q_0q_1
q_1	—	q_2
(q_2)	—	—

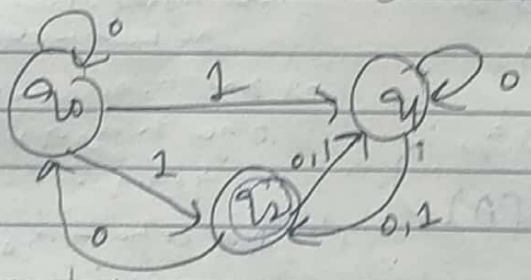
← Transition table

States	a	b
→ q_0	q_0	q_0q_1
q_0q_1	q_0	q_0q_2
(q_0q_2)	q_0	q_0q_2

DFA Transition table

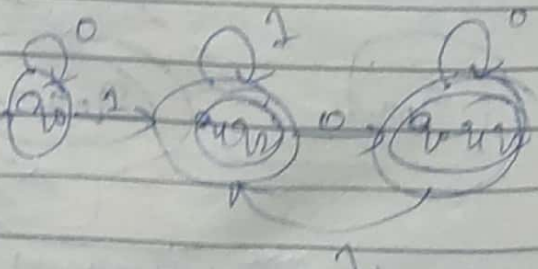


Q.



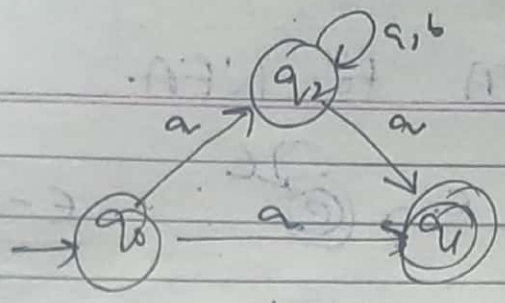
	0	1
→ q_0	q_0	q_1q_2
q_1	q_1q_2	q_2
(q_2)	q_0q_1	q_1

States	0	1
→ q_0	q_0	q_1q_2
(q_1q_2)	$q_0q_1q_2$	q_1q_2
($q_0q_1q_2$)	$q_0q_1q_2$	q_1q_2



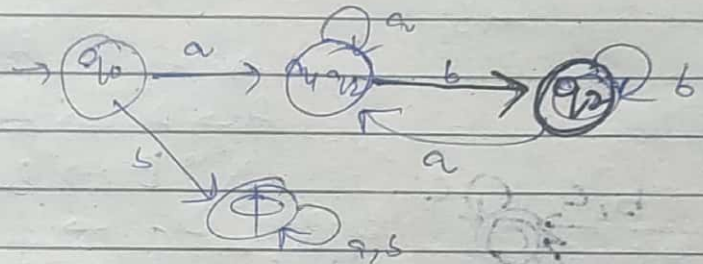
Date : / /
Page No.

Q.

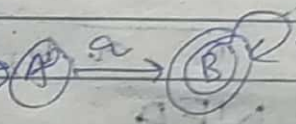


	a	b
→ q_0	$q_2 q_1$	–
q_1	–	–
q_2	$q_1 q_2$	q_2

	a	b
→ q_0	$q_1 q_2$	ϕ
$q_1 q_2$	$q_1 q_2$	q_2
q_2	$q_1 q_2$	q_2
ϕ	ϕ	ϕ



Q. L_1 = starts with 'a'.

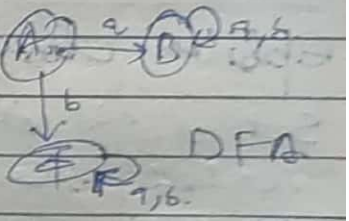


	a	b
A	B	–
B	B	B

NFA

	a	b
A	B	ϕ
B	B	B

DFA



DFA

Q All strings ends with 'a'.

Q All strings in which 2nd symbol from R.H.S is 'a'.

Q 3rd symbol from R.H.S is 'a'.

Q

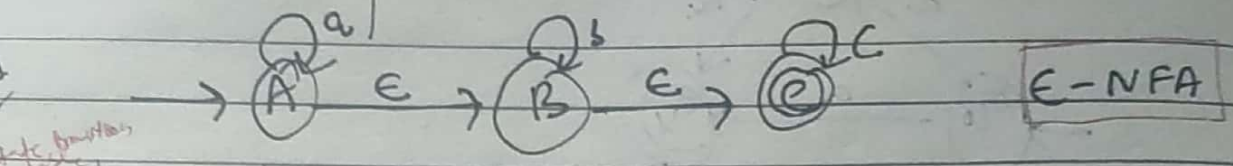
Q

→ Every state on ϵ goes to 'itself' also.

Date: / /
Page No.

Conversion of ϵ-NFA to NFA.

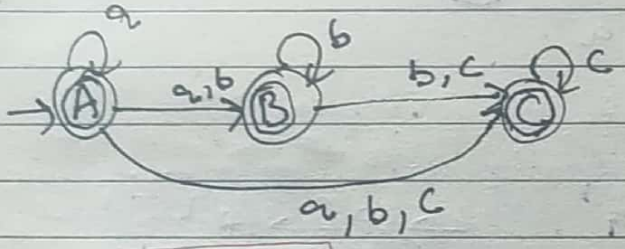
Q)



ϵ-NFA

State transition table

	a	b	c
A	{A,B,C}	{B,C}	{C}.
B	{ϕ}	{B,C}	{C}.
C	{ }	{ }	{C}.



NFA

Epsilon closure / ϵ*:

Set of all states which can be reached from some state.

(a)

→ All the states that can be reached from a particular state, only by seeing ϵ symbol.

$$\delta'(q_i, x) = \epsilon\text{-closure}\left[\delta\left[\epsilon\text{-closure}(q_i), x\right]\right]$$

1. ϵ* a ϵ*

A → A → A → A
B → ϕ → B
C → ϕ → C

2. ϵ* b ϵ*

A → A → ϕ
B → B → B
C → ϕ → C

3. ϵ* c ϵ*

A → A → ϕ
B → ϕ
C → C → C.

4. ϵ* a ϵ*

B → B → ϕ
C → ϕ

5. ϵ* b ϵ*

B → B → B → B
C → ϕ → C

6. ϵ* c ϵ*

B → B → ϕ
C → C → C.

7. ϵ* a ϵ*

C → C → ϕ

8. ϵ* b ϵ*

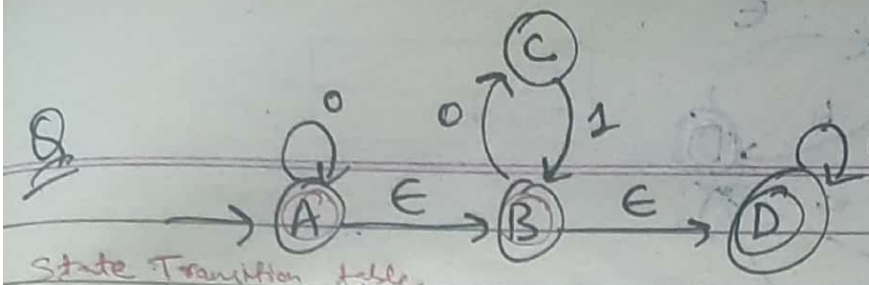
C → C → ϕ.

9. ϵ* c ϵ*

C → C → C → C

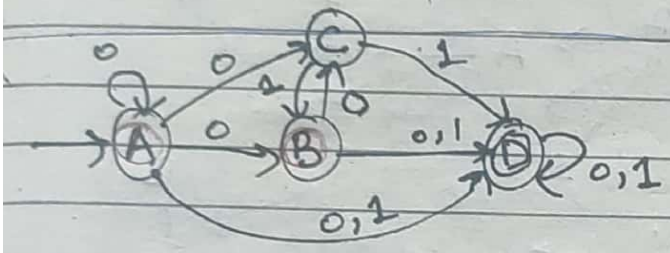
Date : / /
Page No.

Q.



State Transition table.

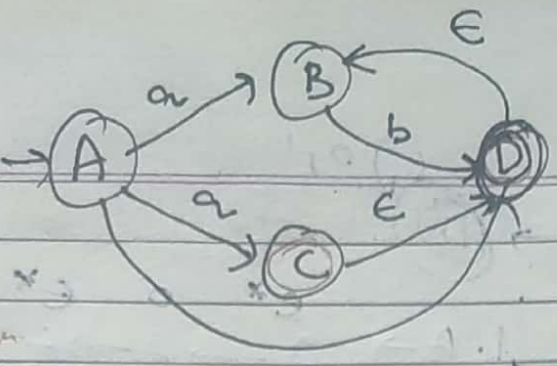
	0	1
A	{A,B,C,D}	{D}
B	{C,D}	{D}
C	ϕ	{B,D}
D	{D}	{D}



1. A → ϵ^* 0 ϵ^*
   A → A → A
   B → C → B
   D → D → C
   D
2. A → ϵ^* 1 ϵ^*
   A → ϕ
   B → ϕ
   D → D → D
3. B → ϵ^* 0 ϵ^*
   B → C → C
   D → D → D
4. B → ϵ^* 1 ϵ^*
   B → B → ϕ
   D → D → D
5. C → ϵ^* 0 ϵ^*
   C → C → ϕ
6. C → ϵ^* 1 ϵ^*
   C → C → B → B
   D
7. D → ϵ^* 0 ϵ^*
   D → D → D → D
8. D → ϵ^* 1 ϵ^*
   D → D → D → D

Date: / /
Page No:

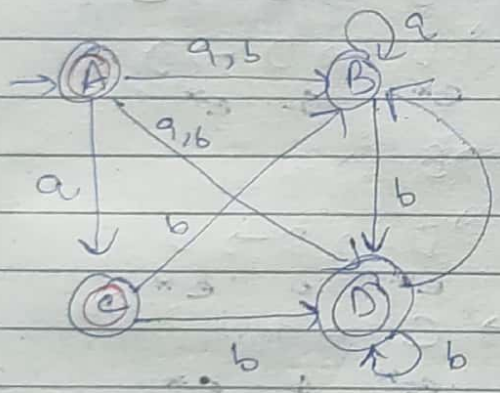
Q



State Transition Diagram

	a	b
A	BCD	BD
B	-	BD
C	-	BD
D	-	BD

NFA



Now Note:- which states to make a final states.
→ States from which by reading ε we reach to final state. Make all those states final.
eg here only A and C can be made final states.

1. $A \xrightarrow{\varepsilon^*} A \xrightarrow{a} B \xrightarrow{\varepsilon^*} B$
   $A \to C \to C$
   $A \to D \to \phi \to B$
   $A \to B \to \phi$

2. $A \xrightarrow{\varepsilon^*} A \xrightarrow{b} \phi \xrightarrow{\varepsilon^*} B$
   $A \to D \to \phi \to B$
   $A \to B \to D \to D$
   $\to B$

3. $B \xrightarrow{\varepsilon^*} B \xrightarrow{a} \phi \xrightarrow{\varepsilon^*} B$

4. $B \xrightarrow{\varepsilon^*} B \xrightarrow{b} D \to D$
   $\to B$

5. $B \xrightarrow{\varepsilon^*} B \xrightarrow{a} \phi \xrightarrow{\varepsilon^*} B$
   $B \to D \to \phi$
   $B \to B \to \phi$

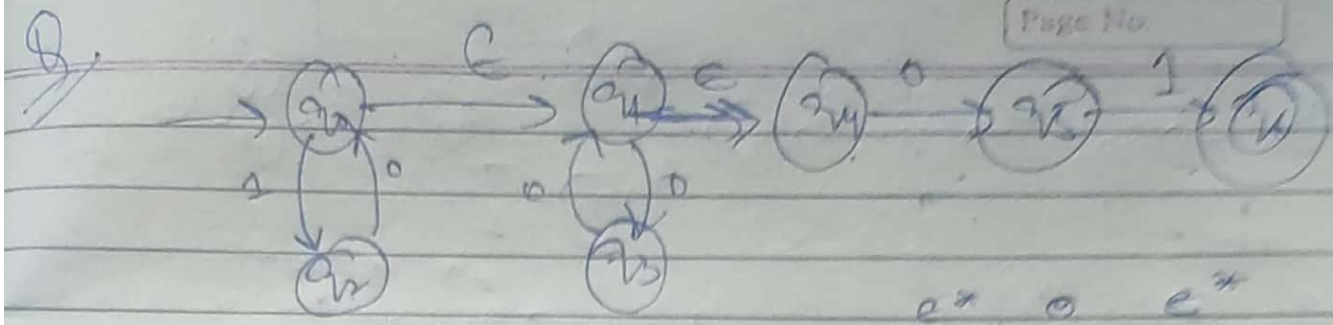
6. $C \xrightarrow{\varepsilon^*} C \xrightarrow{b} \phi \xrightarrow{\varepsilon^*} B$
   $C \to D \to \phi$
   $C \to B \to D \to D$
   $\to B$

7. $D \xrightarrow{\varepsilon^*} D \xrightarrow{a} \phi \xrightarrow{\varepsilon^*} B$
   $D \to B \to \phi$

8. $D \xrightarrow{\varepsilon^*} D \xrightarrow{b} \phi \xrightarrow{\varepsilon^*} B$
   $D \to B \to D \to D$
   $\to B$

Date : / /
Page No

Q.



	0	1
q_0	q_3, q_5	q_2
q_1	q_3, q_5	ϕ
q_2	q_2, q_4	ϕ
q_3	q_4	ϕ
q_4	q_5	ϕ
q_5	ϕ	q_6
q_6	ϕ	ϕ

e^* 0 e^*

$q_0 \to q_0 \to \phi$

$\to q_1 \to q_3 \to$

$\to q_4 \to q_5 \to$

q_0 e^* 1

$q_0 \to q_1 \to$

$\to q_4 \to \phi$

$\to q_2 \to \phi$

$q_1 \to q_1 \to q_3 \to$

$\to q_4 \to q_5 \to$

$q_1 \to q_4 \to \phi \to \phi$

$q_4 \to \phi \to \phi$

$q_2 \to q_2 \to q_2 \to$

$\to q_4$

$\to q_4$

$q_3 \to q_3 \to q_4$

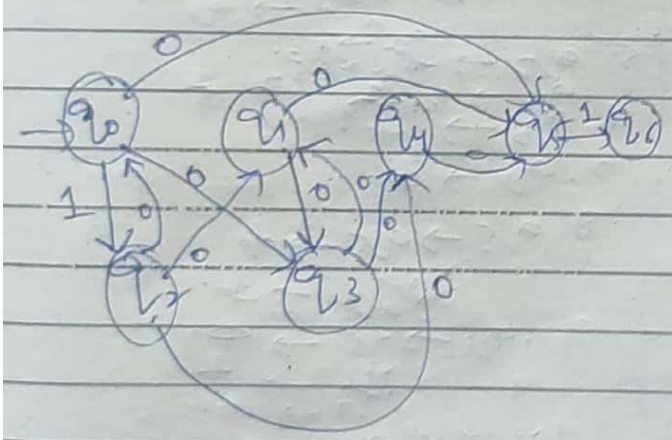
$q_3 \to q_3 \to \phi$

$q_4 \to q_4 \to q_5 \to$

$q_4 \to q_4 \to \phi$

$q_5 \to q_5 \to \phi$

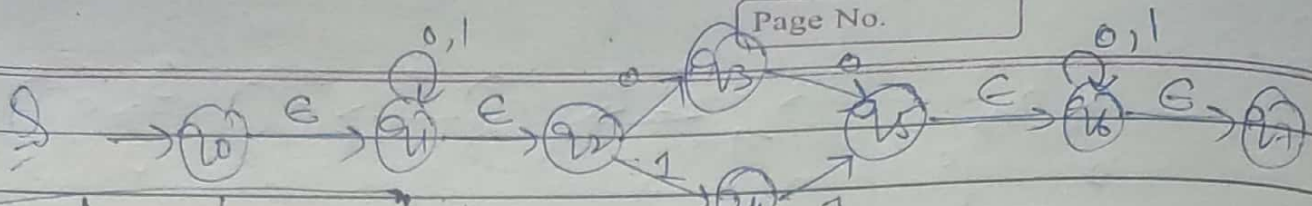
$q_5 \to q_5 \to q_6 \to$



$q_6 \to q_6 \to \phi$

$q_6 \to q_6 \to \phi$

Date : / /
Page No.



	0	1
→ q_0	$q_1 q_2 q_3$	$q_4 q_5 q_6$
q_1	$q_1 q_2 q_3$	$q_4 q_5 q_6$
q_2	q_3	q_4
q_3	$q_7 q_5 q_6$	ϕ
q_4	ϕ	$q_5 q_6 q_7$
q_5	$q_5 q_7$	$q_6 q_7$
q_6	$q_7 q_7$	$q_6 q_7$
q_7	ϕ	ϕ

$\varepsilon^* \; 0 \; \varepsilon^*$

1. $q_0 \to q_0 \to \phi$
   $\to q_1 \to q_4 \to q_1, q_2$
   $\to q_2 \to q_3 \to q_3$
2. $q_1 \to q_1 \to \phi$
   $q_1 \to q_4 \to q_1, q_2$
   $q_2 \to q_4 \to q_4$
3. $q_1 \to q_1 \to q_1 \to q_1, q_2$
   $\to q_2 \to q_3 \to q_3$
4. $q_4 \to q_4 \to q_1, q_2, q_4$
5. $q_2 \to q_3 \to \phi$
6. $q_3 \to \phi$
7. $q_4 \to \phi$
8. $q_5 \to q_5 \to \phi$
   $\to q_6 \to \phi$
   $\to q_7 \to \phi$
9. $q_5 \to q_5 \to \phi$
   $\to q_4 \to \phi$
   $\to q_7 \to \phi$
10. $q_6 \to q_6 \to \phi$
    $\to q_7 \to \phi$
11. $q_7 \to q_7 \to \phi$

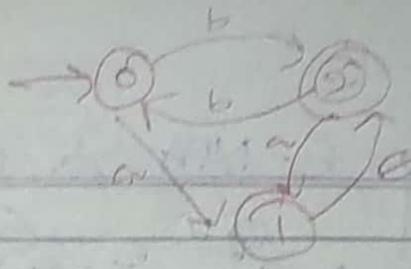
$q_2 \to q_2 \xrightarrow{0} q_3$
$q_2 \to q_2 \xrightarrow{1} q_4$
$q_3 \to q_3 \xrightarrow{0} q_5, q_6, q_7$
$q_3 \to q_3 \xrightarrow{1} \phi$
$q_4 \to q_4 \xrightarrow{0} \phi$
$q_4 \to q_4 \xrightarrow{1} q_5, q_6, q_7$

$q_5 \to q_5 \xrightarrow{0} \phi$
$\to q_6 \to q_6$
$\to q_7 \to \phi$
$q_5 \xrightarrow{1} q_5, q_6, q_7$

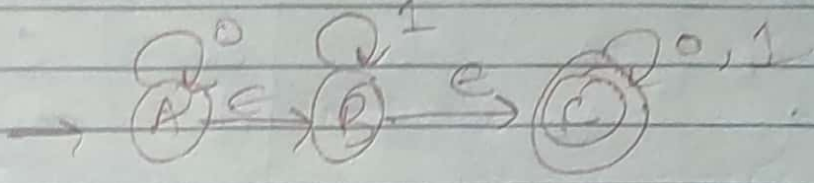
$q_6 \to q_6 \xrightarrow{0} \phi$
$\to q_7 \to \phi$
$q_6 \xrightarrow{1} q_7 \to \phi$
$\to q_6, q_7 \to \phi$

q_7

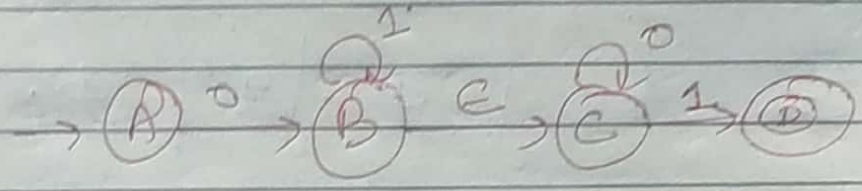
Q1



Q2



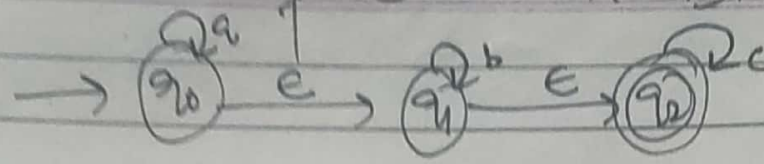
Q3



Q2

Date : / /
Page No.

Conversion of ϵ-NFA to DFA



ϵ-NFA:

δ_ϵ	a	b	c	ϵ
→ q_0	q_0	ϕ	ϕ	q_1
q_1	ϕ	q_1	ϕ	q_2
(q_2)	ϕ	ϕ	q_2	ϕ

Here first we make a transition table for NFA.
→ Then find ϵ-closure of all states.
→ Then draw DFA according to the rule
1. Make initial state of ϵ-NFA's ϵ-closure as initial state.
→ Then find Union of ϵ-closure of all those states & then again find ϵ-closure of that newly state
→ Repeat same for all states till every state will be explored.
→ At last construct DFA from the above DFA table

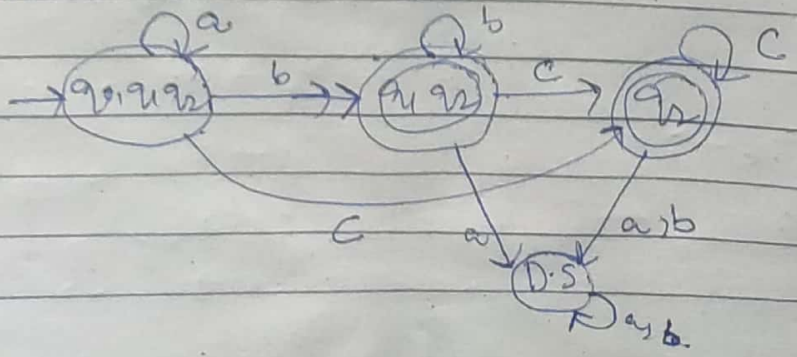
ϵ-closure of (q_0) = q_0, q_1, q_2
ϵ-closure of (q_1) = q_1, q_2
ϵ-closure of (q_2) = q_2.

DFA

DFA state δ_D	a	b	c
→ $\{q_0 q_1 q_2\}$	$q_0 q_1 q_2$	q_1, q_2	q_2
($q_1 q_2$)	ϕ	q_1 / q_2	q_2
(q_2)	ϕ	ϕ	q_2

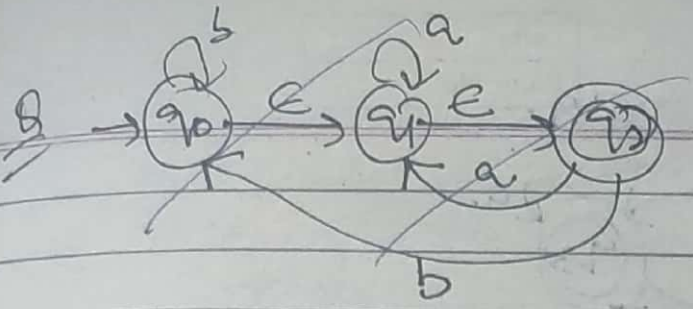
$\delta_D(\{q_0 q_1 q_2\}, a) = \epsilon\text{-closure}(\delta_\epsilon(q_0 q_1 q_2\}, a))$
$= \epsilon\text{-closure}(\delta_\epsilon(q_0, a) \cup \delta_\epsilon(q_1, a) \cup \delta_\epsilon(q_2))$
$= \epsilon\text{-closure}(q_0 \cup \phi \cup \phi)$.
$= \epsilon\text{-closure}(q_0)$.
$= q_0, q_1, q_2$

Repeat the same step.

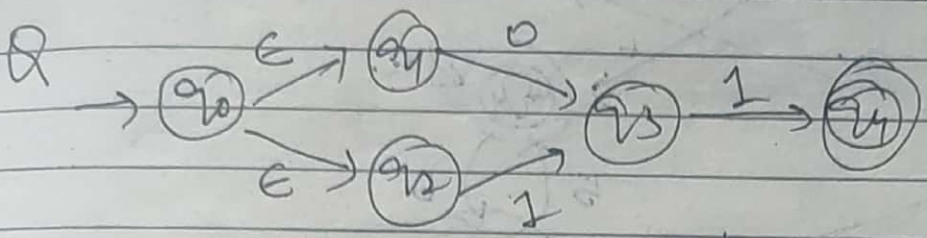


Date : / /
Page No. 1

Q



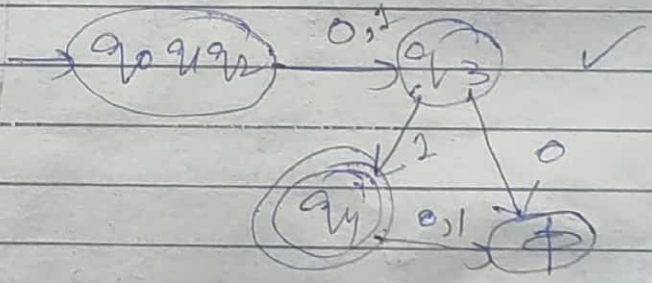
Q



ε-closure of q_0 = q_0, q_1, q_2.
ε-closure of q_1 = q_1.
ε-closure of q_2 = q_2.
ε-closure of q_3 = q_3.
ε-closure of q_4 = q_4.

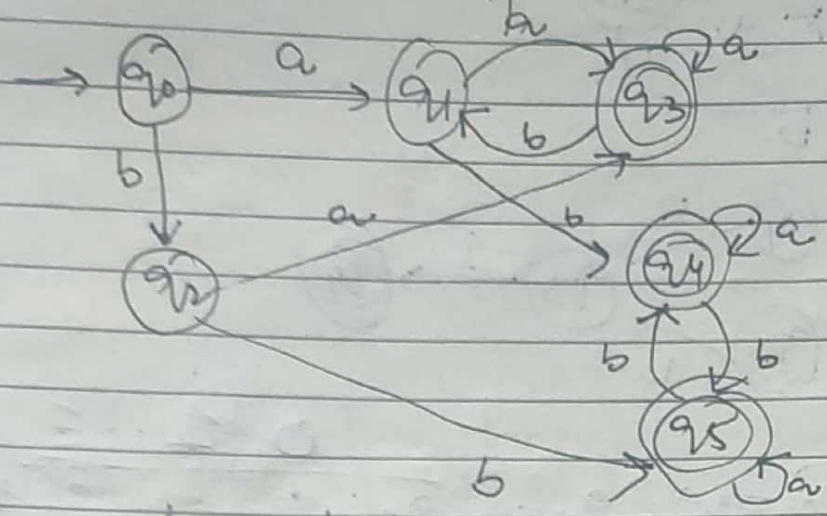
	q_0	q_1	ε
q_0	ϕ	ϕ	q_1 q_2
q_1	q_3		
q_2		q_3	
q_3		q_4	
q_4			

	0	1
→ q_0 q_1 q_2	q_3	q_3
q_3	ϕ	q_4
* q_4	ϕ	ϕ



Date : / /
Page No.

Minimization of DFA



	a	b
→ q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_3	q_5
q_3	q_3	q_4
q_4	q_4	q_5
q_5	q_5	q_4

1. Remove all the states that are unreachable from initial state.
2. Draw transition diagram

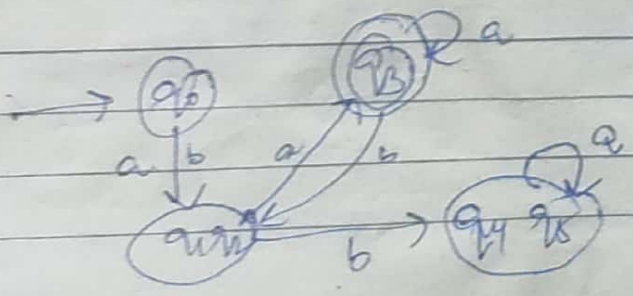
$\pi_0 = \{q_0 q_1 q_2\}\ \{q_3 q_4 q_5\}$

q_{11} q_{12}

$\pi_1 = \{q_0\}\ \{q_1\ q_2\}\ \{q_3\}\ \{q_4\ q_5\}$

q_{11} q_{12} q_{13} q_{14} q_{21} q_{22}

$\pi_2 = \{q_0\}\ \{q_1 q_2\}\ \{q_3\}\ \{q_4 q_5\}$



Date : / /
Page No.

The process of elimination of states whose presence or absence doesn't affect the language accepting & the result is minimal capacity or capability of DFA is called minimization of automata. & the result is minimal DFA. or commonly called as MFA (minimal F.A). MFA is always unique for a language.

→ It is sometimes difficult to design a minimal DFA directly, so a better approach is to first design DFA & then minimized

→ Based on Productivity, states of DFA can be mainly classified in two types.

a) productive states: State is said to be productive, if it adds any accepting power to the machine that is its presence & absence effect the lang. accepting capability of machine

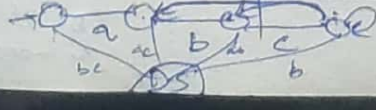
b) Non productive states: These states don't add anything to the language accepting power to the machine.

→ They can be divided into 3 types.

a) Dead state (TT) Unreachable state (UU) Equal state

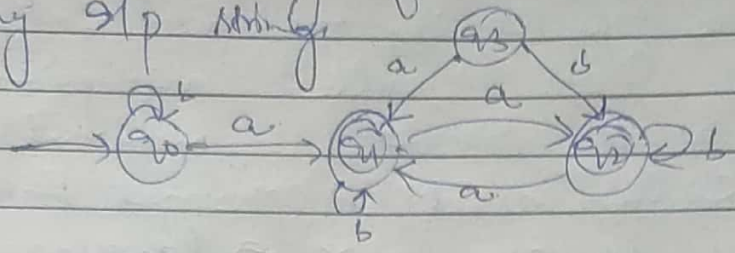
a) Dead states: It is basically created to make the system complete, can be defined as a state from which there is no transition possible to the final state.

→ In DFA there can be more than one dead state but logically always one dead state is sufficient to complete the functionality



Date : / /
Page No.

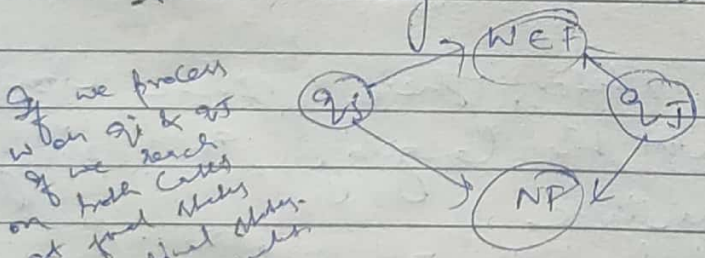
Unreachable States: It is that state which can't be reached starting from initial state by passing any i/p string.



Equal states: These are those states that behave in same manner on each & every i/p string that is for any string w where $w \in \Sigma^*$ either both of the states will go to final state or both will go to non-final state (remember the eg of equal state DFA).

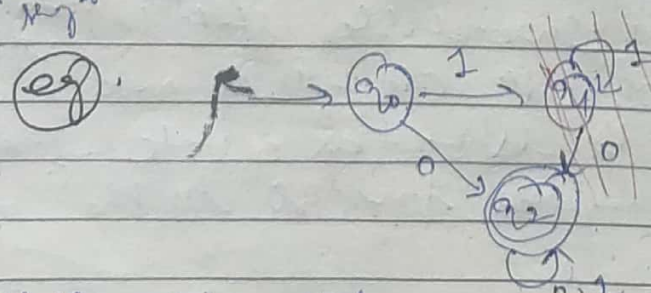
→ More formally two states q_1 & q_2 are equivalent (denoted by $q_1 \equiv q_2$) if both $\delta(q_1, x)$ & $\delta(q_2, x)$ are final states or both of them are non final states for all $x \in \Sigma^*$.

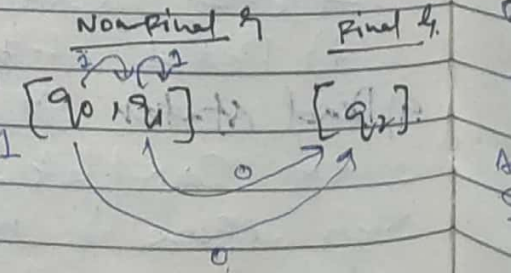
If q_1 & q_2 are k- equivalent for all $k \geq 0$ then they are k- equivalent.



If we process w on q_i & we reach to one of the final states or non final states respectively then

eg.



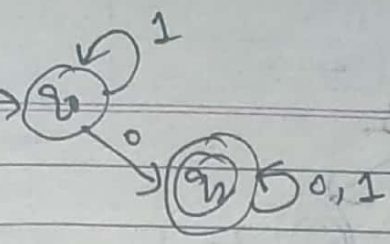


(3) all the outgoings will be removed as well. But the incomings are made as self loop for the state where it was coming from.

(0) → so we can say $q_0 \& q_1$ are equal states

(1) → so we can remove one of them, Now q_0 is initial, so we will remove q_1 —

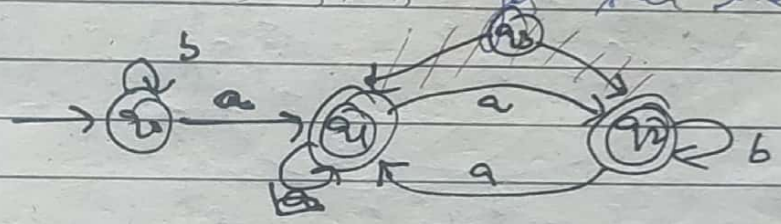
L = → every string which contains at least one zero.



Procedure for Minimization of DFA

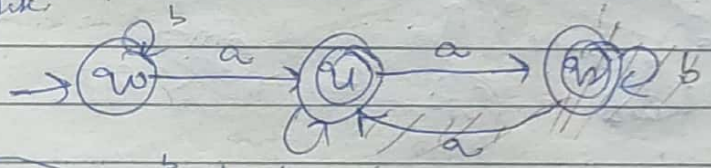
1) For this first of all, group all non-final states in one set & all final states in another set.
2) Now, on both the sets, individually check, whether any of the underlying elements (states) of that particular set are behaving in same way, that is are they having same transition (to same set) on each i/p alphabet
3. If the ans. is yes then the 2 states are eqvl otherwise not

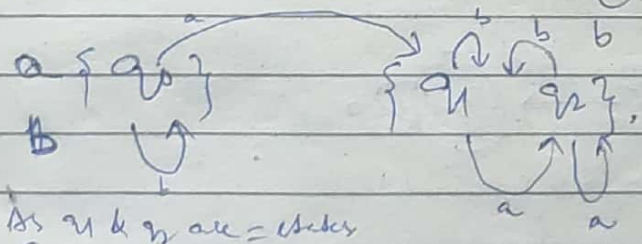
Q.



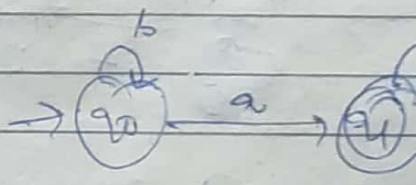
Step 1. Find dead state: No.
Step 2 Find U. Reachable state, Yes.
then eliminate all U.R.S.

Step 3: Now find Eqvl state
Divide in 2 grps.
F & N.F states



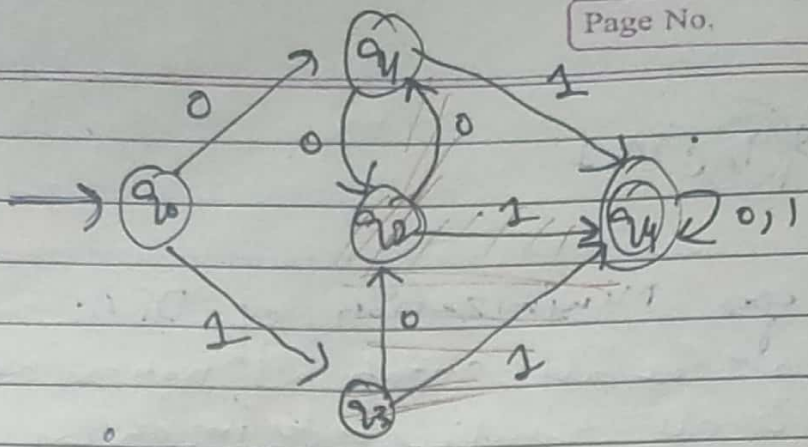


As q_1 & q_2 are = states
So remove q_2 & all its outgoing.

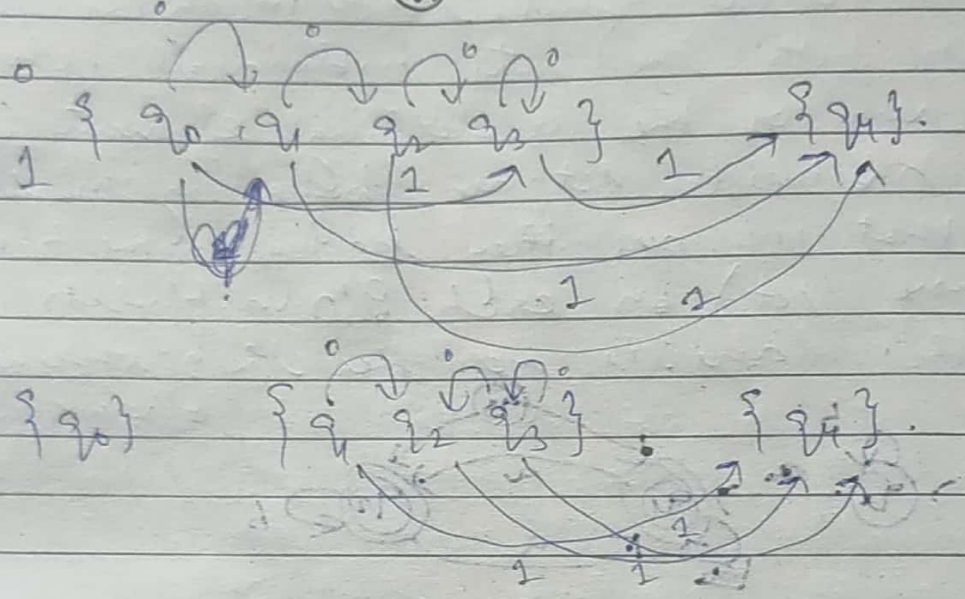


Date : / /
Page No.

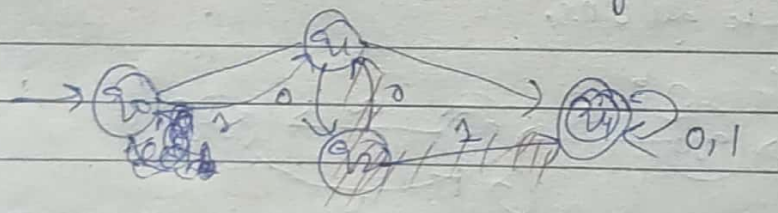
Q)

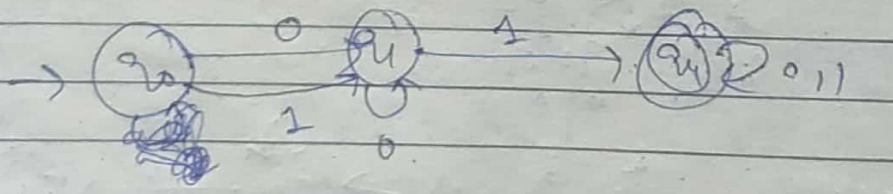


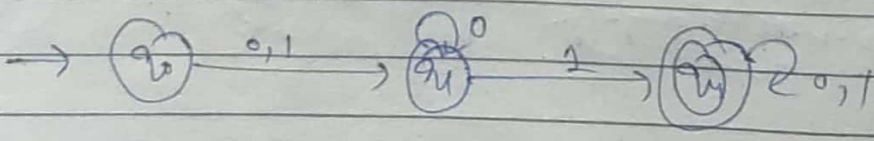
Find D-S(TM)
Find N.R.S(TM)
Find B.S

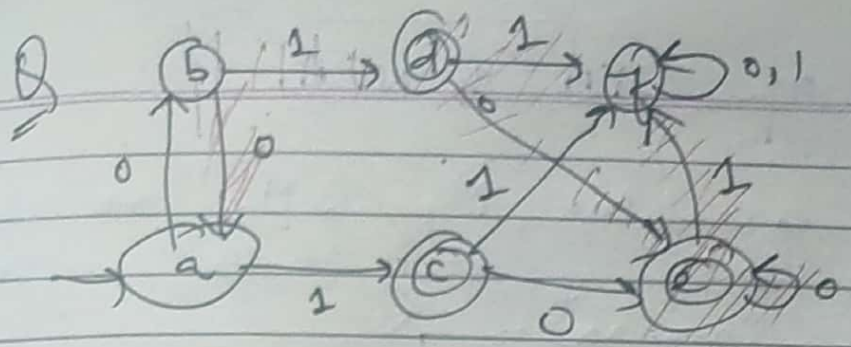


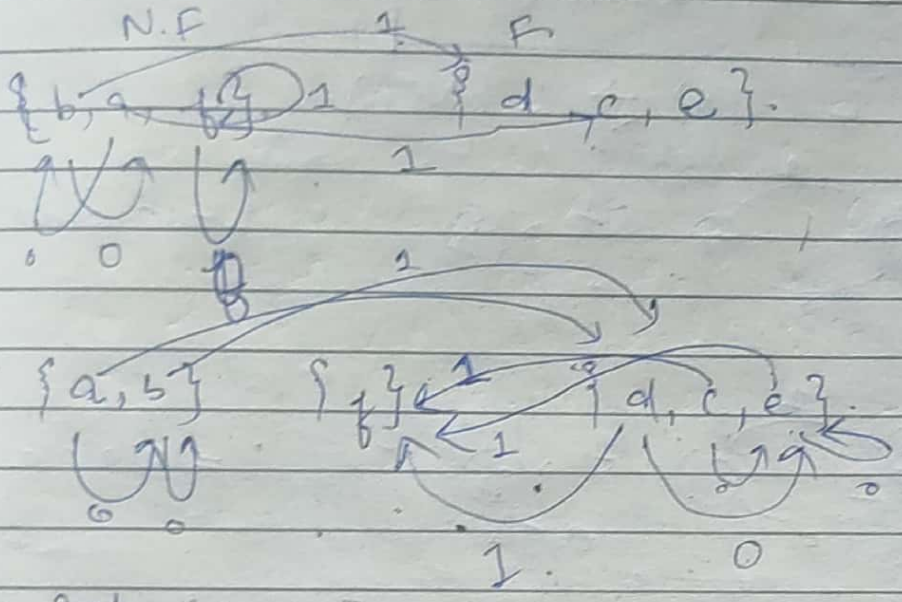
q1 q2 q3 equal states.
to remove q2 q3 & its outgoings,
& all their incomings will go to the state which was equal to removed states.
i.e incomings will come to q1.









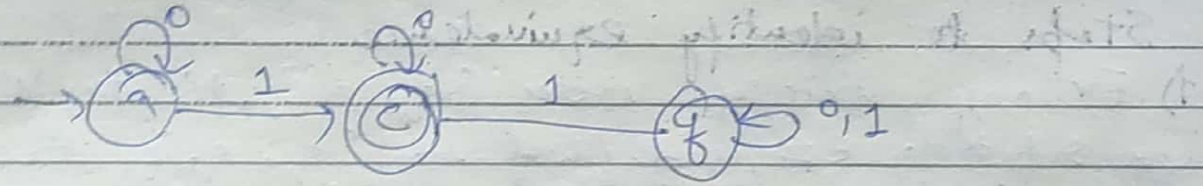


q2b.

remove b.

d=c=e.

remove d and e



Date: / /
Page No.

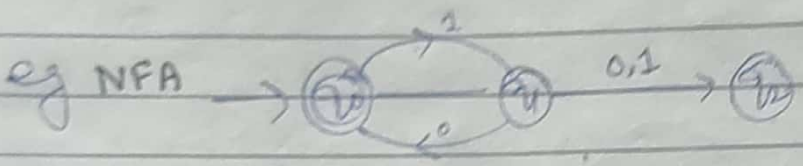
Equivalence of DFA & NFA.

Two finite acceptors, M_1 & M_2 are said to be equivalent if they both accept the same language.

i.e $L(M_1) = L(M_2)$.

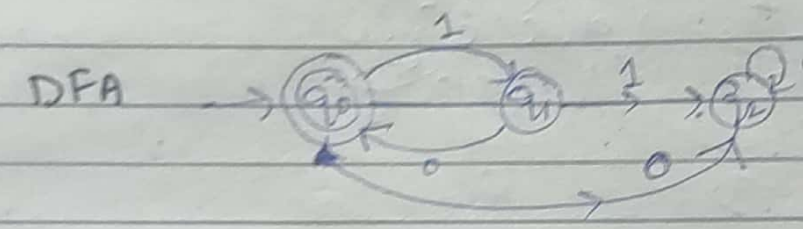
$L(DFA) \rightleftharpoons L(NFA)$

eg NFA



$L = 10101010$

DFA



$L = 10101010$

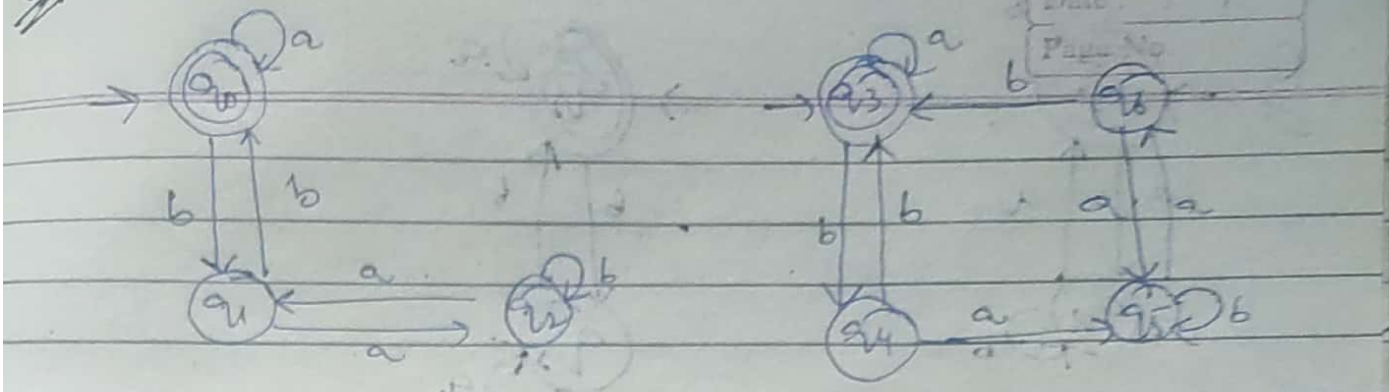
$L = \{(10)^n : n \geq 0\}$

Steps to identify equivalence

1) For any pair of states $\{q_i, q_j\}$, the transition for input $a \in \Sigma$ is defined by $\{q_a, q_b\}$ where $\delta\{q_i, a\} = q_a$ and $\delta\{q_j, a\} = q_b$.
The two automata are not equivalent if for a pair $\{q_a, q_b\}$ one is INTERMEDIATE state and other is FINAL state.

2) If initial state is Final state of one automaton, then in second automaton also initial state must be final state for them to be equivalent.

Q.

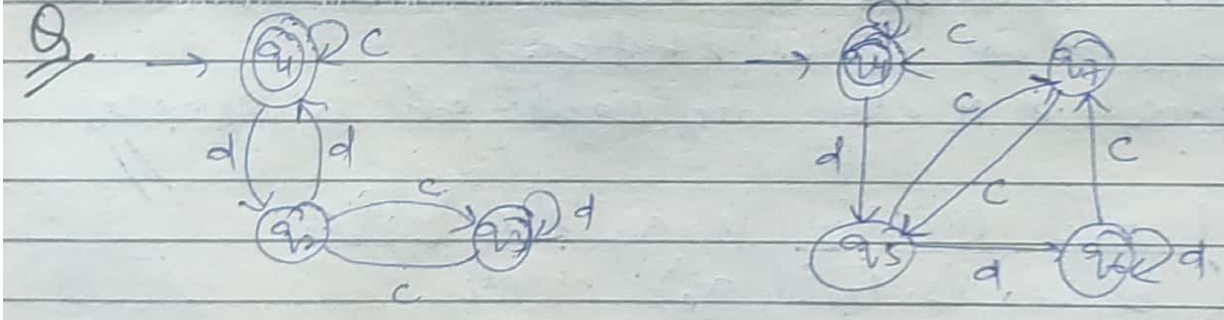


Step(1) Initial & final state of both automata should be same.

states	Input a	Input b
→ ~~$(q_0 q_2)$~~		
→ $(q_0 q_3)$	$(q_1 q_3)$ F.S F.S	$(q_0 q_1)$ I.S I.S
$(q_1 q_4)$	$(q_2 q_5)$ I.S I.S	$(q_0 q_3)$ F.S F.S
$(q_2 q_5)$	$(q_4 q_6)$ I.S I.S	$(q_2 q_5)$ I.S I.S
$(q_4 q_6)$	$(q_2 q_5)$ I.S I.S	$(q_0 q_3)$ F.S F.S

∴ FA 1 = FA 2.

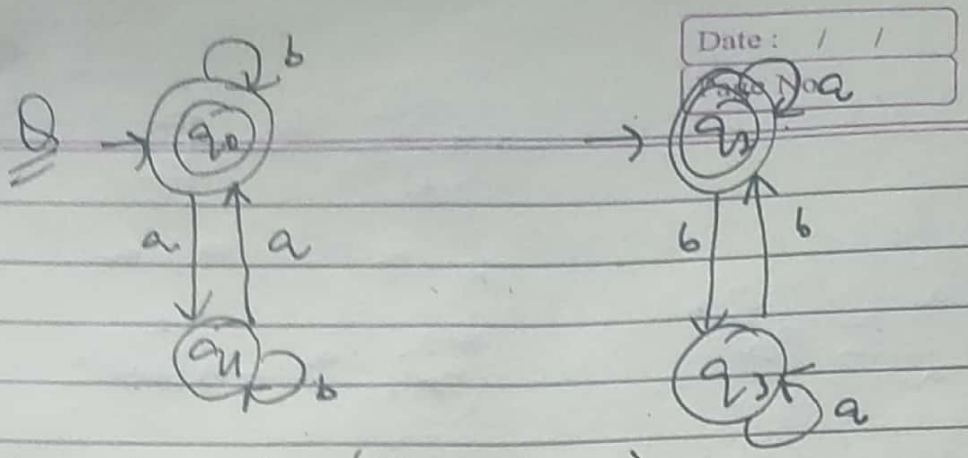
Q.



	c	d
→ $(q_1 q_4)$	$(q_4 q_4)$ F.S F.S	$(q_2 q_5)$ I.S I.S
$(q_2 q_5)$	$(q_3 q_3)$ I.S I.S	$(q_4 q_6)$ F.S I.S

→ Here q_4 & q_6 are F.S & I.S in these two automata are not equivalent

Date : / /



FA with o/p: (Transducers).

* Moore Machine :- Moore Machine is a FSM in which next symbol is decided by current state & current i/p symbol.
- The o/p symbol at a given time depends only on present state of the machine.

→ (Edward F Moore) (George H Mealy).

Both Moore & Mealy machine are special case of DFA.

→ Both acts like o/p producers rather than language acceptors.

→ In Moore & Mealy Machine no need to define final states.

→ No concept of dead states & no concept of final states.

→ Moore & Mealy are equivalent in power.

* Moore Machine is a six tuple
$(Q, \Sigma, \Delta, \delta, \pi, q_0)$ where

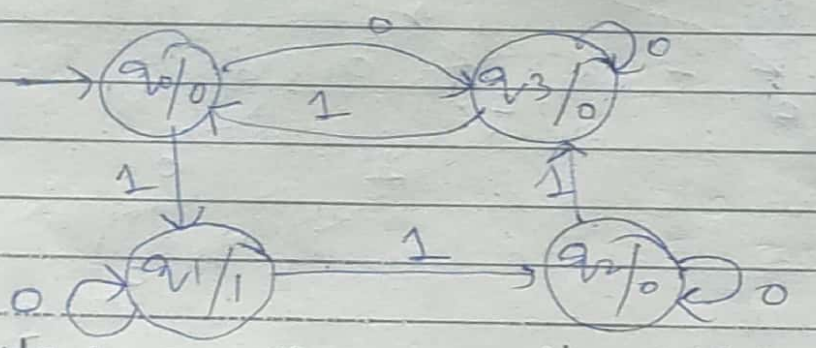
Q = finite no. of states.
Σ = i/p alphabet.
Δ = o/p alphabets
δ = transition fun $\delta: Q \times \Sigma \to Q$.
π = o/p fun $(\lambda: Q \to \Delta)$ mapping Q into Δ.
q_0 = initial state

→ In Moore machine for every state output is associated.

- If length of i/p string is n, then length of o/p string will be n+1.
- Moore machine responds for empty string ∈.

Q The below table shows δ table of Moore Machine,

present state	Next state δ		output
	a=0	a=1	
q_0	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

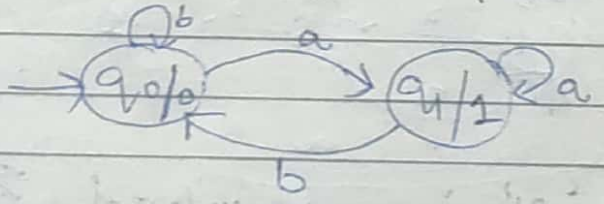


$q_0 q_3 q_2 q_3$

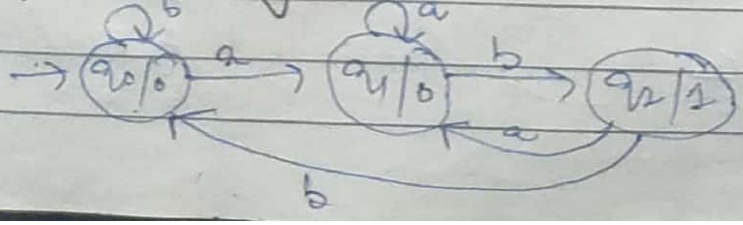
q_0,			
0	1	0	0

i/p = 1101
o/p = 01000

Q Construct a Moore machine that take all the strings of a's & b's as i/p and counts the no. of a's in i/p string in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$?

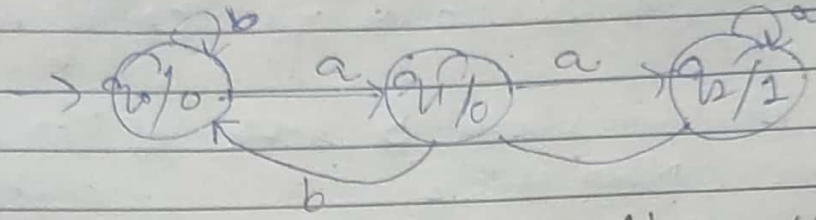


Q. Construct a Moore machine that take all the strings of a's & b's as i/p & counts the no. of occurance of sub string 'ab' in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$

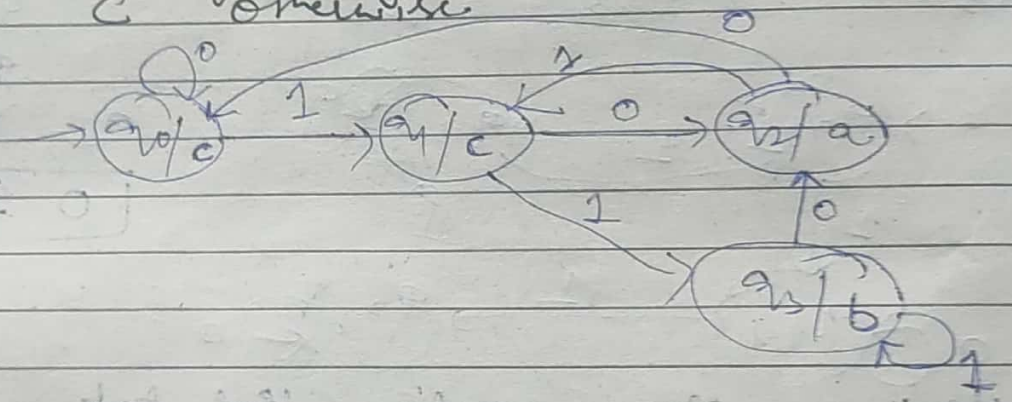


Date : / /
Page No.

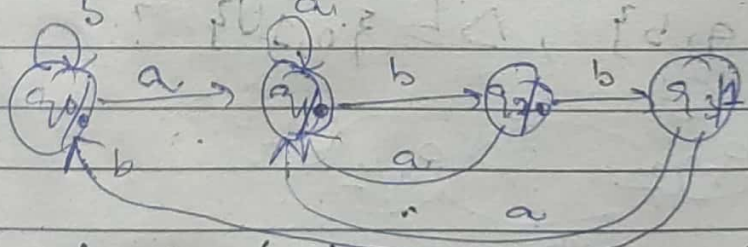
Q. Construct a Moore machine take all string of a's & b's as i/p & count the no. of occurance of sub-string 'aa' in terms of 1.
$\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$.



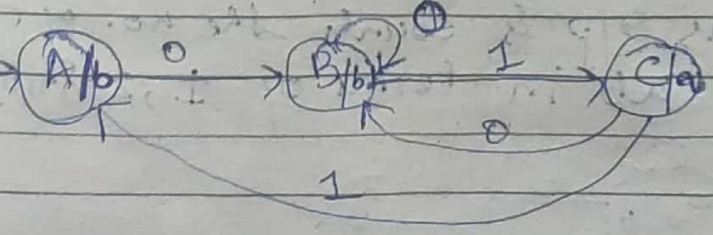
Q Construct a Moore machine where $\Sigma = \{0,1\}$
$\Delta = \{a, b, c\}$, machine should give
o/p 'a' if i/p string ends with 10, ——10 a
'b' if i/p string ends with 11, ——11 b
'c' otherwise c



Q count abb, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$.



Q Print 'a' whenever '01' is encountered. $\Sigma = \{0,1\}$, $\Delta = \{a, b\}$



check for 0110

0 1 1 0
A B C A B
b b a b b

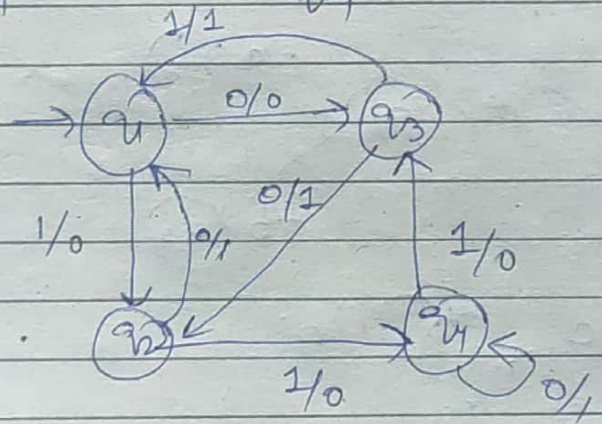
Date : / /
Page No.

Mealy Machine.

- Improved version of Moore machine.
- Mealy machine is a six tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where all symbols except λ have same meaning as in Moore machine.
- λ is o/p function mapping $Q \times \Sigma$ into Δ.
- In case of mealy machine o/p symbol depends on transition.

Eg. Below table shows transition tab of Mealy Machine

pres. state	Next state, $a=0$	o/p	Next state, $a=1$	o/p
→ q_1	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0



o/p = 1011

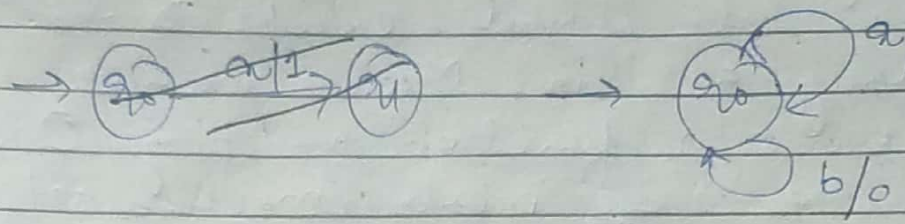
	q_2q_1	q_2q_4
q_1	X0	XX

0100 o/p.

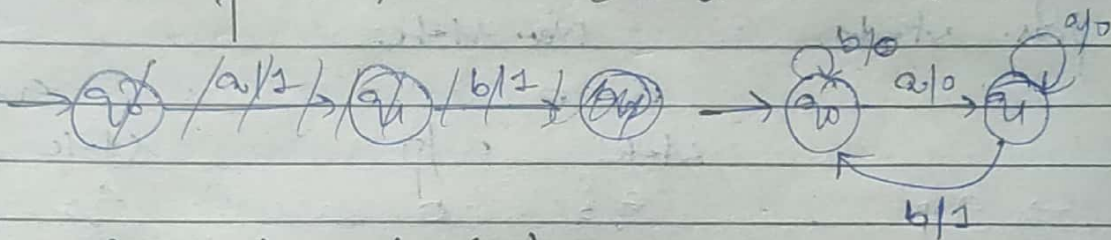
Note:
- If length of i/p string is n, then length of o/p string will be n.
- Mealy machine also not response for empty string.

Date : / /
Page No.

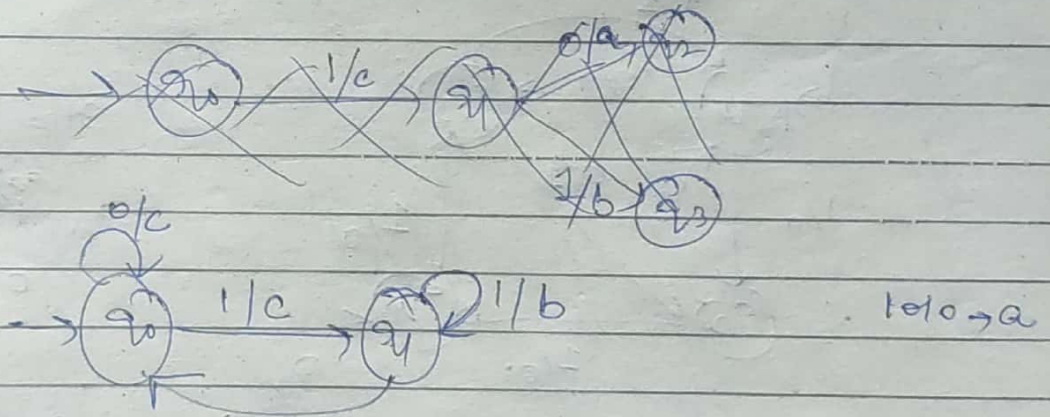
Q Count no. of a's in i/p string in terms of 1,
$\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$.



Q Count no. of occurance of sub string 'ab' in terms of 1, $\Sigma = \{a, b\}$, $\Delta = \{0, 1\}$.



Q Machine should give o/p 'a' if i/p string ends with 10,
'b' if i/p string ends with 11,
'c' otherwise.



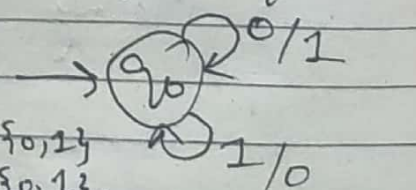
1010 → a

Q To find 1's Complement & design Mealy machine

Most significant bit

1 0 1 0 0 ← least significant bit

0 1 0 1 1 — 1's Complement



$\Sigma = \{0, 1\}$
$\Delta = \{0, 1\}$.

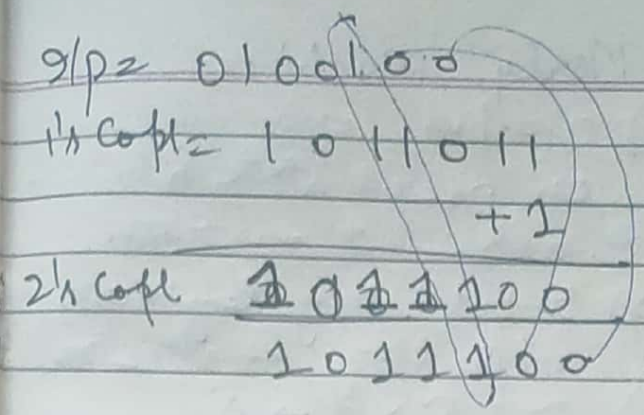
Q 2's Complement

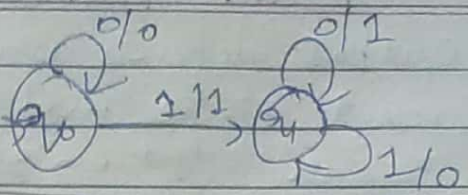
1's Compl + 1

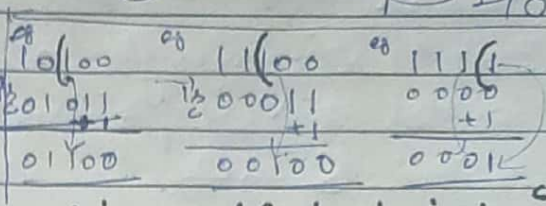
i/p = 0 0 1 0 1
1's Compl = 1 1 0 1 0
+ 1
1 1 0 1 1

Q. Construct a Mealy machine that gives 2's complement of any binary string. (Assume that last carry bit is neglected).

Date : / /
Page No.

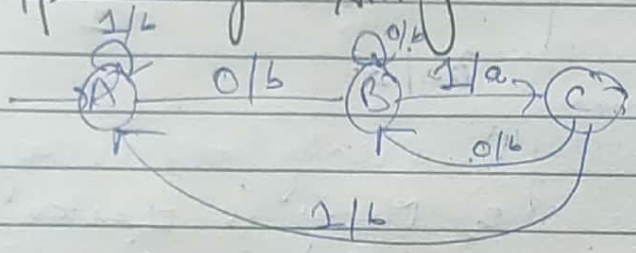






Q. Construct a Mealy machine that prints "a" whenever sequence 01 is encountered in any o/p binary string.

$\Sigma = \{0, 1\}$
$\Delta = \{a, b\}$.

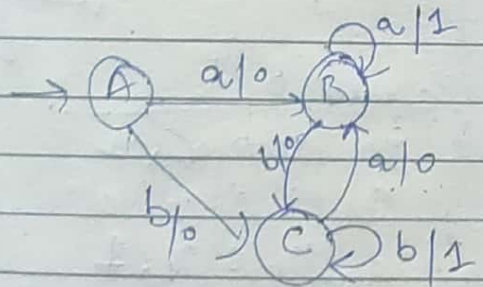


Q. Construct a Mealy machine that produces 1's Complement of any binary o/p string.

Q. Construct a Mealy machine that print "a" whenever sequence 01 is encountered in any o/p Binary string.

Q. Construct a Mealy machine accepting a language consisting of strings from Σ^*, where $\Sigma = \{a, b\}$ & string should end with aa or bb.

aa → 1
bb → 1

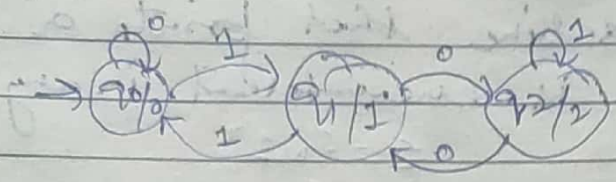


Q. Construct

Date: / /
Page No.

Moore Machine

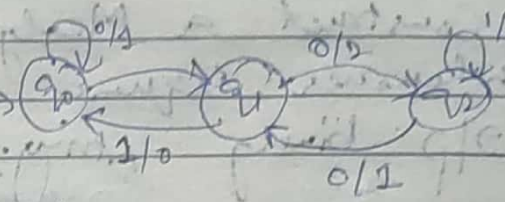
1) Output depends on present state and is independent on i/p.
2) Input string length = n
output string length = n+1
3) $\lambda : Q \to \Delta$



4) Output is synchronous with clock
5)

Mealy Machine

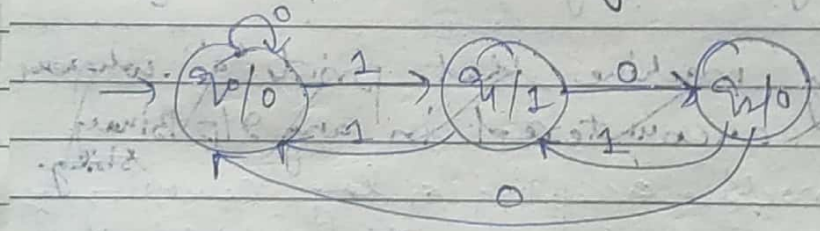
1) Output only depends on present state & pres. i/p & independent on next i/p.
2) Input string length = n
o/p string length = n.
3) $\lambda : Q \times \Sigma \to \Delta$.



BS/3.

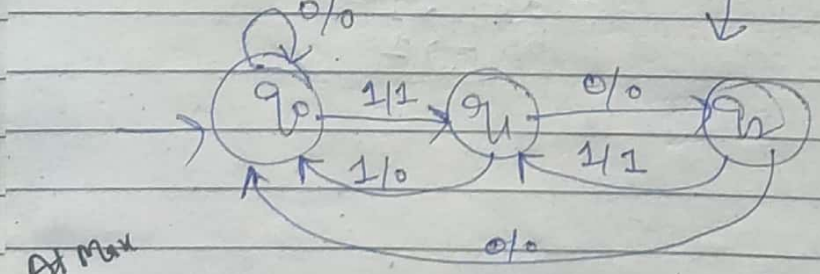
4) Mealy o/p is asynchronous.
5) Mealy is faster than Moore

Moore to Mealy Conversion.



States	Next state 0	Next state 1	output
q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_0	q_1	0

Convert



Convert

states	0	1
q_0	q_0,0	q_1, 1
q_1	q_2,0	q_2, 0
q_2	q_0,0	q_1, 1

At Max

Same states

m states, n output = Max m-states.

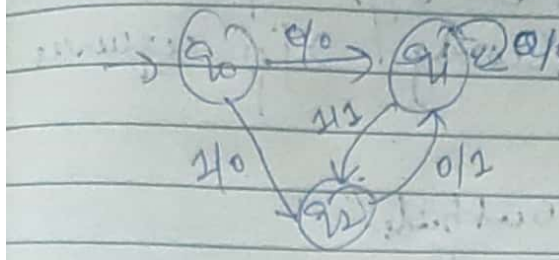
while Converting Moore to Mealy

while Conversion, first draw all 3 states q_0, q_1, q_2 then see where transition goes to, write o/p of that state with transition. eg q_0 goes to q_1 on 1 & q_1 has 1 o/p, so write 1/1 on arrow.

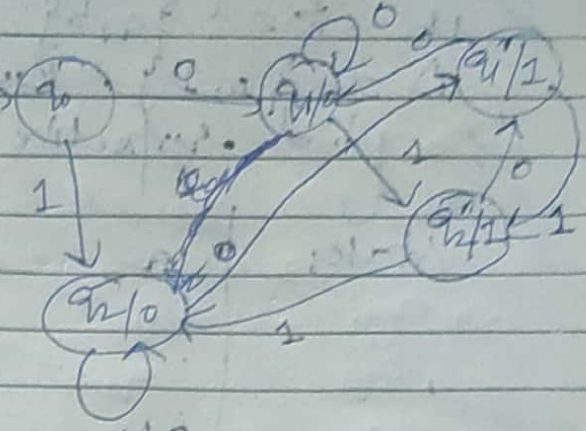
Here all current states as it is, in the column of 0 & 1 as it is, then check for q_0 has output 0 in Moore so write that along with q_0 in column under 0, as $(q_0, 0)$. in Mealy.

Date : / /
Page No.

Mealy to Moore Conversion



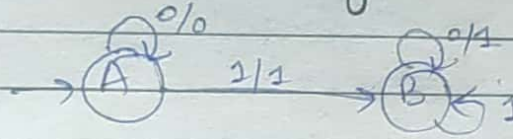
3 states.



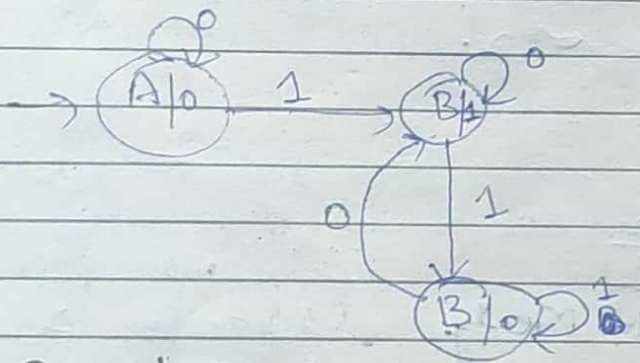
5 states.

	at max.
min	n states, n outputs → min. states
mxn	3 2 → 3×2=6 states
	Mealy → Moore

Q Convert Mealy to Moore

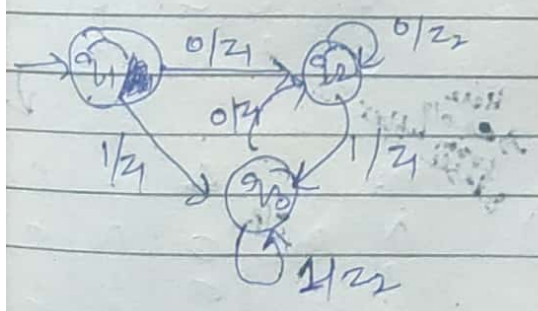


P.S	0	Δ	1	Δ
→ A	A	0	B	1
B	B	1	B	0

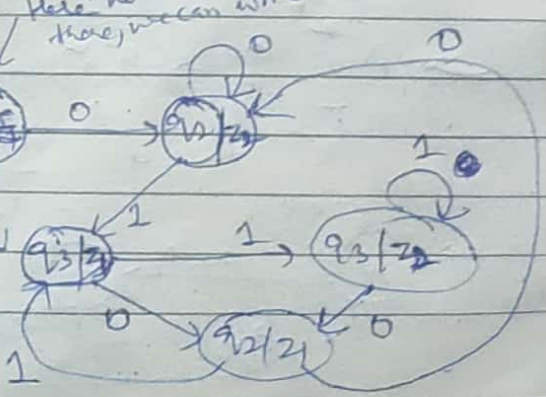


P.S	0	1	Δ
→ A	A	B	0
B	B	B	

Q Convert Mealy to Moore



Here is no incoming edge, so no problem here, we can write ε here or blank.



Date : / /
Page No.

Q For the following Moore machine i/p alphabet is $\Sigma = \{a, b\}$ and the o/p alphabet $\Delta = \{0, 1\}$. Run following i/p sequences & find respective o/p.

States	a	b	outputs.
→ q_0	q_1	q_2	0
q_1	q_2	q_3	0
q_2	q_3	q_4	1
q_3	q_1	q_4	0
q_4	q_0	q_0	0

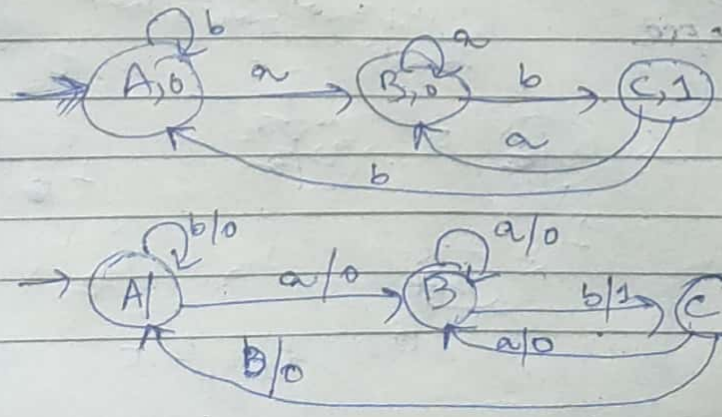
(i) aabaab (ii) abbb (iii) ababb

(i) a a b a b
q_0 q_1 q_2 q_4 q_0 q_2
0 0 1 0 0 1 = o/p

(ii) a b b b
q_0 q_1 q_3 q_4 q_0
0 0 0 0 0 = o/p.

(iii) a b a b b
q_0 q_1 q_3 q_4 q_0 q_2
0 0 0 0 0 1

Q2 Convert Moore into Mealy



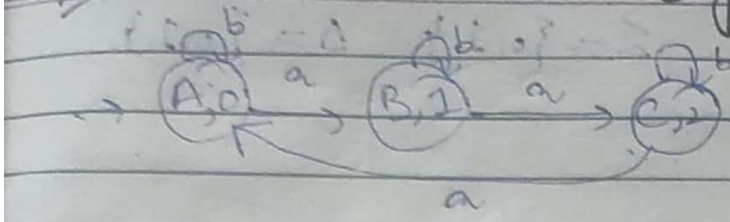
P.S	N.S a	N.S b	o/p
→ A	B	A	0
B	B	C	0
C	B	A	1

Here o/p is written from state

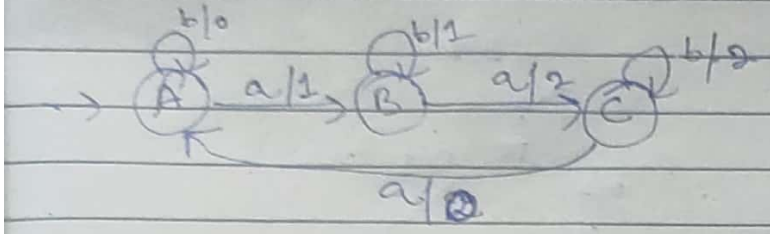
P.S	a	o/p	b	o/p
A	B	0	A	0
B	B	0	C	1
C	B	0	A	0

Date : / /
Page No:

Q. Convert Moore into Mealy



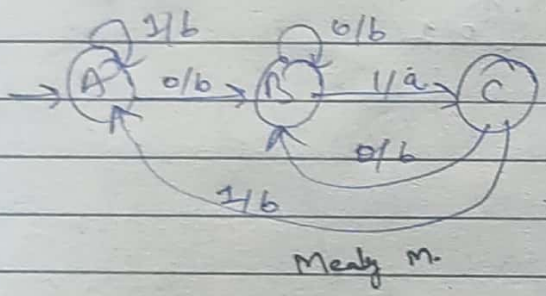
P.S	N.S a	N.S b	Δ
→ A	B	A	0
B	C	B	1
C	A	C	2



	a		b	
	N.S	Δ	N.S	Δ
→ A	B	1	A	0
B	C	2	B	1
C	A	0	C	2

Same o/p from upper & table

Q Construct a Moore machine that prints 'a' whenever the sequence '01' is encountered in any i/p binary str & then convert it to its equivalent Mealy machine.

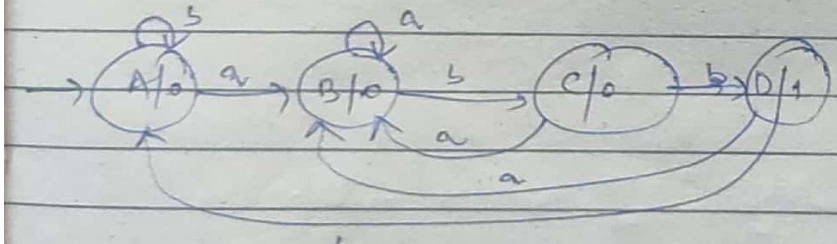


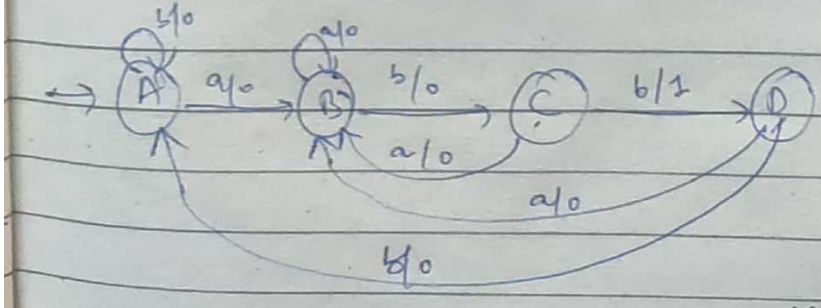
Mealy M.

P.S	0	1
→ A	B, b	A, b
B	B, b	C, a
C	B, b	A, b

Mealy M δ-table of Mealy machine

Q The given Moore machine counts occurrence of seq 'abb' in any i/p string over {a,b}. Convert to its equivalent Mealy machine
Σ = {a,b}, Δ = {0,1}.





Mealy diag

	a	b
A	B, 0	A, 0
B	B, 0	C, 0
C	B, 0	D, 1
D	B, 0	A, 0

Mealy STable

Date : / /
Page No.

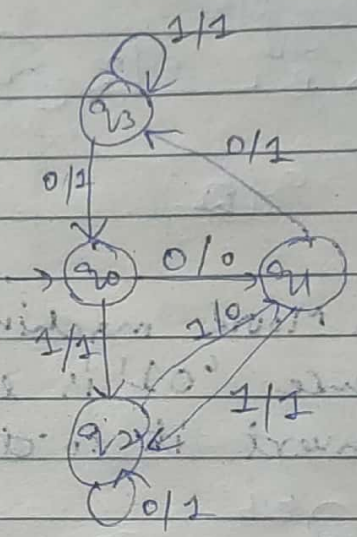
Q. Convert the given Moore Machine to its equivalent Mealy machine. $\Sigma = \{0,1\}$ $\Delta = \{0,1\}$.

Moore

State	0	1	o/p
→ q_0	q_1	q_2	1
q_1	q_3	q_0	0
q_2	q_2	q_1	1
q_3	q_0	q_3	1

Mealy

State	0	1
→ q_0	q_1, 0	q_2, 1
q_1	q_3, 1	q_2, 1
q_2	q_2, 1	q_1, 0
q_3	q_0, 1	q_3, 1



Date : / /
Page No.

Comparison = finite then we can make machines & infinite then not.

Regular languages.

1) $L = \{a^m b^n \mid m, n \geq 0\}$ ✓
2) $L = \{a^m b^n c^q \mid m, n, q \geq 0\}$ ✓
3) $L = \{a^{x_1}, b^{x_2}, \ldots z^{x_{26}} \mid x_i \geq 0 \mid 1 \leq i \leq 26\}$ ✓
4) $L = \{a^m b^n \mid 1 \leq m < 50, 1 \leq n \leq 1000\}$ ✓
5) $L = \{a^n b^n \mid 1 \leq n \leq 10\}$ ✓
6) $L = \{a^n b^n \mid 1 \leq n \leq 2^{\text{14th prime no.}}\}$ ✓
7) $L = \{a^n b^n \mid 1 < n \leq 2^{14 \text{ STATE}}\}$ ✓
8) $L = \{a^n b^n \mid n \geq 0\}$ ✗

eg 8

→ If we have to check whether lang is regular or not we will check if we can make machine for that if yes then it is R.L if not then Nb.
→ Whenever in order is to remember finite ~~A~~. Remember
→ and for Comparison F.A will do for Finite L. not for Infinite L.
→ FA can't do Infinite comparison.
→ FA do state change.

(9) $L = \{a^m b^n \mid m > n, n \geq 0\}$ ✗
(10) $L = \{a^m b^n \mid m, n \geq 0, m \neq 0\}$. ✗
(11) $L = \{a^m b^n \mid m$ is div. by $n\}$. ✗
12) $a^m b^n \mid m = n^p \mid p \geq 1\}$. ✗
13) $L = a^m b^n c^q \mid m = n = q\}$. ✗
14) $L = \{a^m b^n c^q \mid m + n = q\}$ ✗
✓15) $L = \{a^m b^n, \mid m + n =$ even$\}$
✓16) $L = \{a^m b^m \mid m + n =$ odd$\}$.
✗(17) $L = \{wcw^R \mid w \in \Sigma^*\}$
✗18) $L = \{wcw \mid w \in \Sigma^*\}$
✗19) $L = \{wcw^R \mid w \in \Sigma^+\}$
✗20) $L = \{wcw \mid w \in \Sigma^+\}$
✗21) $L = \{w.w \mid w \in \Sigma^*\}$
✗22) $L = \{w.w^R \mid w \in \Sigma^+\}$
✓23) $L = \{wcw^R \mid w \in \Sigma^*\}$ ✓

✓24) $L = \{cww^R \mid c, w \in \Sigma^*\}$
✓25) $L = \{ww^R c \mid c, w \in \Sigma^*\}$.

a^0 a^1

→ a way of representing R.L
→ expression of strings and operators.

Date : / /
Page No.

Regular Expressions.

→ Method used to represent a regular language.
- It is used to denote regular languages.
- R.E are mathematical expressions used to represent regular languages.
- R.L are generated by R.E.
- So any lang accepted by F.A can be represented using regular expression.

primitive R.E

- ϕ represents $\{\}$. No empty strings
- ϵ / λ represents $\{\epsilon\}$ zero length.
- a represents $\{a\}$
- b represents $\{b\}$.

If r_1 & r_2 are regular expressions. then

$r_1 + r_2$
$r_1 \cdot r_2$
r_1^*

are also regular expressions.

operations of R.E.

Union = (+) $(a+b)$ (either a or b) (choice)
Concatenation = (·) $(a \cdot b)$
positive closure = a^+
Kleene closure = a^*

Precedence

→ R.E is said to be valid if it can be derived from the primitive R.E. by using these rules. $, r_1^*, r_1^+, r_1 + r_2, r_1 * r_2$.

Date : / /
Page No.

(1) $R = \phi$ $L(R) = \{\}$ / ϕ. = (q0)

(2) $R = \epsilon$, $L(R) = \{\epsilon\}$ = (q0)

(3) $R = a$, $L(R) = \{a\}$ = (q0) —a→ (q1)

(4) $R = a+b$, $L(R) = \{a, b\}$. = (q0) —a,b→ (q1)

(5) $R = a.b$, $L(R) = \{ab\}$. = (q0) —a→ (q1) —b→ (q2)

(6) $R = a+b+c$, $L(R) = \{a, b, c\}$ =

(7) $R = (ab + a).b$, $L(R) = \{abb + ab\}$.

(8) $R = a^*$ $a.L = \{\epsilon, a, aa, aaa, \dots\}$.

$R = a^+$ $L = \{a, aa, aaa, \dots\}$.

$(a+b)^*$ = set of all possible strings containing a & b.

$L = \{\epsilon, a, b, ab, aab, aabb, \dots\}$.

$(a.b)^*$ $L = \{a, b, ab, aab, \dots\}$.

Q Set of all strings whose length is exactly = 2.

$L = \{aa, bb, ab, ba\}$.

$(a+\phi) = a$

$(a+\epsilon) = a, \epsilon$

Regular expression = $(ab + ba + aa + bb)$

$= a(b+a) + b(b+a)$.

$= (a+b).(a+b)$.

$= (a+b)^2$. ✓

(9) $R = (a+ba)(b+a)$, $L(R) = \{ab, aa, bab, baa\}$.

(10) $R = (a+\epsilon)(b+\phi) = (ab, b)$.

(11) $R = (a+b)^2$; $(a+b)(a+b) = (aa, ab, ba, bb)$.

(12) $R = (a+b)^* . (a+b) = (a+b)^*$

$= (\epsilon, a, \dots)(a+b)$

$= (a+b)^+$ → because will concatenate ϵ with any other thing it will not give ϵ.

(13) $R = a^* . a^* = (\epsilon, a, a^2 \dots)(\epsilon, a, \dots a^2) =$

$= (\epsilon, a, a^2 \dots) = a^*$.

(14) $(ab)^* = (\epsilon, ab, \dots, abab$.

Date : / /
Page No.

Design R.E for $\Sigma = \{a, b\}$.

1) Start with ab
$ab(a+b)^*$

2) Start with bba
$bba\cdot(a+b)^*$

3) ends with abb
$(a+b)^*abb$

4) Containing sub string aab
$(a+b)^*aab(a+b)^*$
baa, aaab, aaab, baab, aaabb, baabb

5) Start & ends with a
$a + a(a+b)^*a$
Here we write a at start also bcz it accepts single a also.

6) Start & ends with same symbol
$a + a(a+b)^*a + b(a+b)^*b + b$.

7) Start & ends with diff. symbol
$a(a+b)^*b + b(a+b)^*a$

8) $|w| = 3$.
$(a+b)(a+b)(a+b) = (a+b)^3$

9) $|w| \geq 3$.
$(a+b)^3(a+b)^*$

10) $|w| \leq 3$.
$\epsilon + (a+b) + (a+b)^2 + (a+b)^3$
or $(a+b+\epsilon)^3$

Note:
$ab+bb$
$(a+b)b$ ✓
$b(a+b)$ X. Don't write like this bcz it will give diff strings.
ba+bb is not a given string.

(11) 3rd Symbol from R.H.S is 'a'
$(a+b)^*a(a+b)(a+b)$
(a b a b)

(12) 3rd symbol from L.H.S is 'a'
$(a+b)(a+b)a(a+b)^*$
(a b a)

Date : / /
Page No.

(13) L_1 = even length string
$= \{\epsilon, aa, ab, ba, bb, \ldots\}$.

R.E $= (aa+ab+ba+bb)^*$
or $((a+b)(a+b))^*$
(2 length string)
or $(a+b)^{2*} = (a+b)^{2n}$, $n \geq 0$
(usually we don't write like this.)

(14) L = odd length string
$= \{a, b, aba, bab, aab, bba, baa, abb$

R.E $= ((a+b)(a+b))^*(a+b)$.
for 2 length $+1$.
$= 3$.

$(a+b)^{2n}(a+b)$
or $(a+b)^{2n+1}$.

(15) L = Multiple of 3. or $L \equiv 0 \pmod 3$
0, 3, 6, 9, 12, ..
$((a+b)(a+b)(a+b))^* = ((a+b)^3)^*$

(16) L = set of all strings that contains exactly two a's
$n_a(w) = 2$.
$L = \{aa, baa, bbaa, aba, aabb, \ldots\}$.

$b^*a.b^*a.b^*$

(17) L = at most two a's $n_a(w) \leq 2 = \{0, 1, 2\}$
$0^{a's} \rightarrow b^* = \{\epsilon, b, bb, bbb, \ldots\}$.
$1^{a's} \rightarrow a = b^*ab^*$
$2^{a's} \rightarrow aa = b^*ab^*ab^*$ (Now we will union this)

$(b^* + b^*ab^* + b^*ab^*ab^*)$
(or) $b^*(a+\epsilon)b^*(a+\epsilon)b^*$

Date : / /
Page No.

18) L = Atleast 2 a's. $|n_a(w) \geq 2|$

$(a+b)^* a (a+b)^* a (a+b)^*$

19) L = Even no. of a's or $n_a(w) \bmod 2 = 0$.

$0 \bmod 2 = 0$.

$= \{0, 2, 4, 6, \ldots\}$.

R.E = $b^* + (b^* a b^* a b^*)$

for 0's — for 2 length a's.

20) $n_a(w) \bmod 2 = 1$ (odd no. of a's).

$b^* a b^* (b^* a b^* a b^*)^*$ (or) $(b^* a b^* a b^*)^* b^* a b^*$

1 → 0 = 1
1 → 2 = 3
1 → 4 = 5
1 → 6 = 7

21) L = $n_a(w) \bmod 3 = 0$ $\{0, 3, 6, 9, 12, \ldots\}$

R.E = $b^* + (b^* a b^* a b^* a b^*)^*$.

22) L = $n_a(w) \bmod 3 = 1$ $\{1, 4, 7, 10, 13, \ldots\}$

R.E = $(b^* a b^* a b^* a b^*)^* b^* a b^*$

3
6

23) L = $n_a(w) \bmod 3 = 2$. $\{2, 5, 8, 11, \ldots\}$

RE = $(b^* a b^* a b^* a b^*)^* b^* a b^* a b^*$

0 → 2 = 2
3 → 5 = 5
6 → 2 = 8

$\Sigma = \{a\}$

Date : / /
Page No.

$L = a^n \mid n \geq 0 \mid = a^* = R.E$

$L = \{\epsilon, a, a^2, a^3, \dots\}$.

$L_2 = a^n \mid n \geq 1$ $\quad R.E = a^+$ (or) $a.a^*$ (or) $a^*.a$

$L_2 = \{a, a^2, a^3, \dots\}$

$L_3 = a^{2n} \mid n \geq 0$ $\quad L_3 = \{(a^2)^n = (a.a)^n = (aa)^*\}$

$L_4 = a^{2n+1} \mid n \geq 0$

$= a^{2n} . a$

$= (a.a)^n . a$

$= (a.a)^* . a$

Q. 28th symbol from right end is a

$(a+b)^* a (a+b)^{27}$

Q 4th symbol from right end is b.

$(a+b)^* b (a+b)(a+b)(a+b)$ (or) $(a+b)^* b (a+b)^3$

Q $|w| = 0 \pmod 3 \rightarrow$ (if we divide length of a string by 3 remainder should be 0)

$0, 3, 6, 9, 12, \dots$

$((a+b)^3)^*$

Q $|w| = 2 \pmod 3$ [if we divide length of string by 3 remainder should be 2]

$(2, 5, 8, 11, 14)$

for 2 $(a+b)^2 ((a+b)^3)^*$ for mod 3

Q $|w|_b = 0 \pmod 2$

$a^* (a^* b a^* b a^*)^*$

for 0 for mod 2

Q $|w|_a = 1 \pmod 3$

$b^* a b^* (b^* a b^* a b^* a b^*)^*$

for 1 for mod 3

Q $|w|_b = 2 \pmod 3$

$a^* b a^* b a^* (a^* b a^* b a^* b a^*)^*$

$0^0 =$?

Date: / /
Page No.

Imp. points - 1

1) $R = \epsilon^*$
$L(R) = \{\epsilon^0, \epsilon^1, \epsilon^2, \ldots\}$
$= \{\epsilon\}$.

2) $R = \epsilon^+$
$L(R) = \{\epsilon^1, \epsilon^2, \epsilon^3, \ldots\}$
$= \{\epsilon\}$

3) $R = \phi^*$
$L(R) = \{\phi^0, \phi^1, \phi^2, \ldots\}$
$= \epsilon = \{\epsilon\}$

4) $R = \phi^+$
$L(R) = \{\phi^1, \phi^2, \phi^3, \phi^4, \ldots\}$
$= \phi$.

1) $R = \epsilon^*$, $L(R) = \epsilon$.

2) $R = \epsilon^+$, $L(R) = \epsilon$

3) $R = \phi^*$, $L(R) = \epsilon$

4) $R = \phi^+$, $L(R) = \phi$

(1) $R^+ \cup R^* = R^*$

$(a^1, a^2, a^3, \ldots) \cup (a^0, a^1, a^2, \ldots)$
$= (a, a^2, a^3, \ldots) \cup \{\epsilon, a, a^2, \ldots\}$
$= \{\epsilon, a, a^2, a^3, a^4, \ldots\} = R^*$

(2) $R^* \cdot R^+ = R^+$

$(a^0, a^1, a^2, \ldots) \cdot (a, a^2, a^3, \ldots)$
$= (\epsilon, a, a^2, \ldots)(a, a^2, a^3, \ldots)$
$= (a, a^2, a^3, \ldots) = R^+$

(3) $R^+ \cap R^* = R^+$

$(a^1, a^2, a^3, \ldots) \cap (\epsilon, a^1, a^2, \ldots)$
$= (a^1, a^2, a^3, \ldots) = R^+$

(4) $(R^*)^* = R^*$

$((a^0, a^1, a^2 \ldots))^* = (\epsilon, a, a^2, a^3, \ldots)^*$
$= (\epsilon, a, a^2, a^3, \ldots) = R^*$.

(5) $(R^*)^+ = R^*$

$((a^0, a^1, a^2, \ldots))^+ = (\epsilon, a, a^2, \ldots)^+$
$= (\epsilon, a, a^2, \ldots)$

(6) $(R^+)^* = R^*$

$((a^1, a^2, a^3, \ldots))^* = (a^0, a^1, a^2, \ldots)$
$= (\epsilon, a, a^2, a^3, \ldots) = a^*$

(7) $((R^*)^+)^* \cdot R^+ = R^+$

$((R)^*)^+)^* \cdot R^+$

$R^* \cdot R^+ = R^+$

(whenever there is * in power we will get always R^*.)

$(R^*)^{+****} = R^*$.

Date : / /
Page No.

(8) $(a+b)^* = (a^*+b)^*$

(9) $(a+b)^* = (a+b^*)^*$

(10) $(a+b)^* = (a^*+b^*)^*$

(11) $(a+b)^* \neq (a.b)^*$

$(a+b)^*$: $\epsilon, a, b, ab, ba, bb, aa, \ldots$

$(a.b)^*$: $\epsilon, ab, (ab)^2, (ab)^3, \ldots$

(12) $(a+b)^* \neq (a^*.b)^*$

$(a+b)^*$: $\epsilon, a, b, ab, ba, bb, aa, \ldots$

$(a^*.b)^*$: $= (a^*b)^* (a^*b)^* (a'b)^* (a'b)'$
$\Rightarrow (\epsilon b)^* (ab)^* (\epsilon b)' (ab)$
$\Rightarrow (b)^* \epsilon (b)' (ab)$.
$\Rightarrow \epsilon, \epsilon, b, ab$.

It will generate single a also

It will not generate single a

(13) $(a+b)^* \neq (a.b^*)^*$

same

It will generate single b.

It will not generate single b.

(14) $(a+b)^* = (a^*b^*)^*$ ✓

a

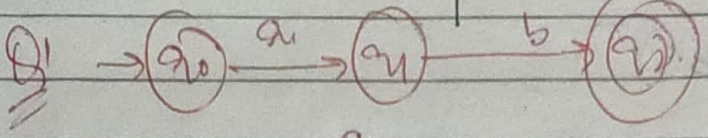
$(a^*b^*)^*$
$\Rightarrow (a^*b^*)^* (a'b')^* (a^*b')^* (a'b^*)^*$
$= (a^*b^*)' (a'b')' (a^*b')' (a'b^*)'$
$\Rightarrow (\epsilon)^* (ab)^* (\epsilon b)^* (a\epsilon)^* (\epsilon)' (ab)' (\epsilon b) (a\epsilon)'$
$\Rightarrow \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, ab, b, a$ ✓

Note:
- For Union we don't need internal star, if we have external star it represents Universal set.
- For Concatenation, if we need to make a power of Concatenation Universal, we need to make all internal elements having power star as seen in (14).

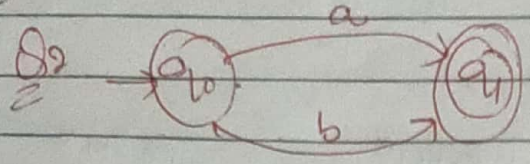
* we write Union in parallel & concatenation in series.

Date : / /
Page No.

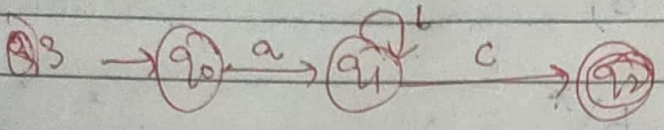
Conversion of F.A to R.E

Q1 

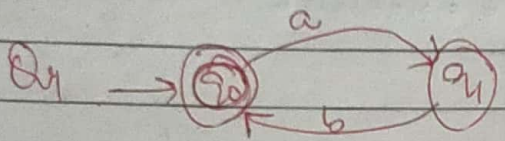
R.E = ab.

Q2 

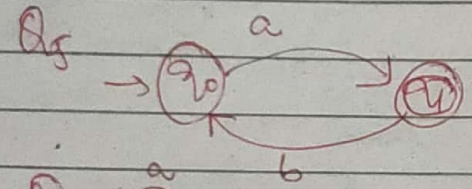
R.E = a+b

Q3 

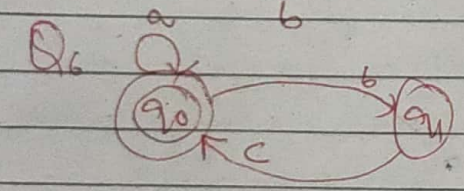
R.E = ab^*c

Q4 

R.E = $(a.b)^*$

Q5 

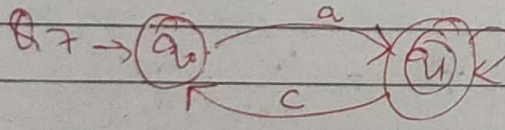
R.E = $a(ba)^*$

Q6 

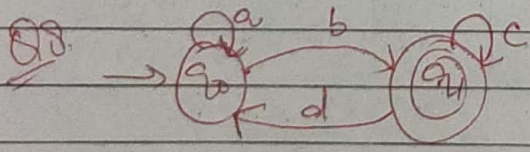
R.E = $(a^*+(bc)^*)^*$

$(a+bc)^*$

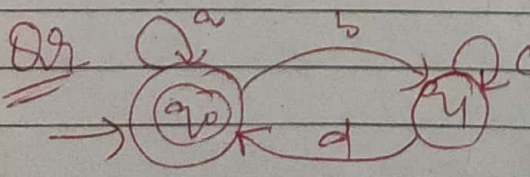
As we know $(a^*+b^*)^* = (a+b)^*$

Q7 

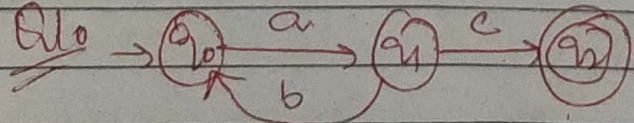
R.E = $a(b+ca)^*$

Q8 

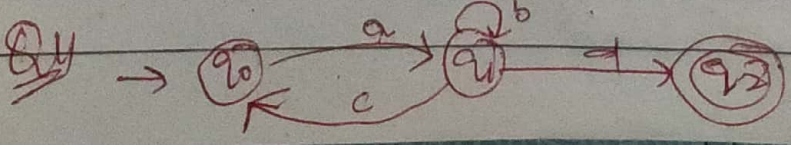
R.E = $a^*b(c+da^*b)^*$

Q9 

R.E = $(a+bc^*d)^*$

Q10 

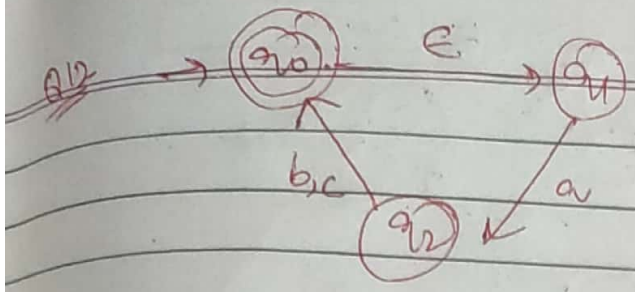
R.E = $a(ba)^*c$

Q11 

R.E = $a(b+a)^*d$

Date : / /
Page No.

Q12

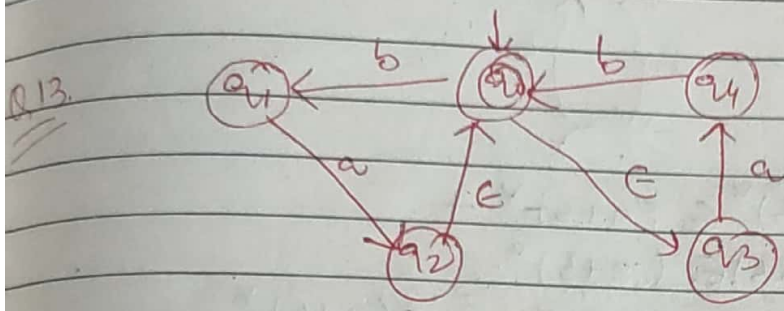


$R.E = (\epsilon \cdot a(b+c))^*$

(or)

$(a(b+c))^*$.

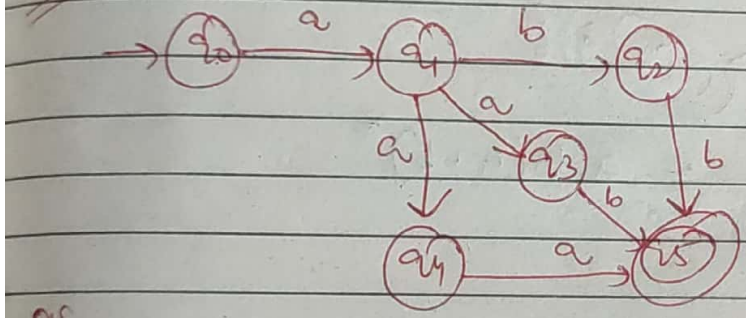
Q13.



$R.E = ((ba\epsilon)^* + (\epsilon ab)^*)^*$

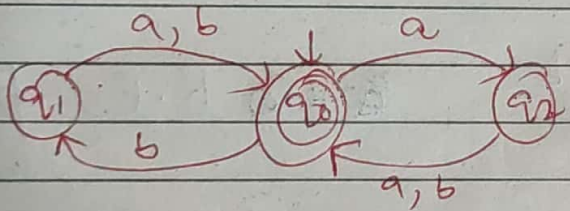
$R.E = (ba + ab)^*$

Q14.



$R.E = a(bb + aa + ab + aa)$

Q15

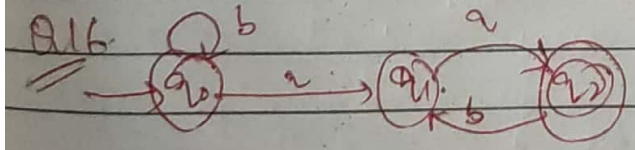


$R.E = (a(a+b) + b(a+b))^*$

$= ((a+b)(a+b))^*$

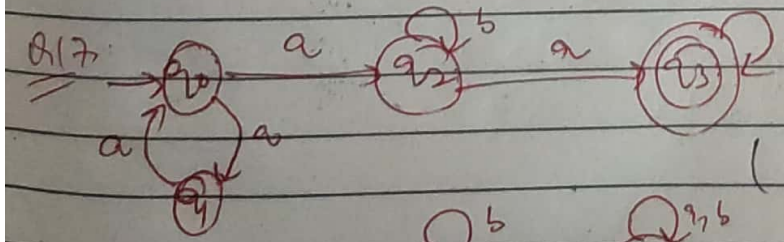
$= ((a+b)^2)^*$

Q16.



$R.E = b^* a a (ba)^*$

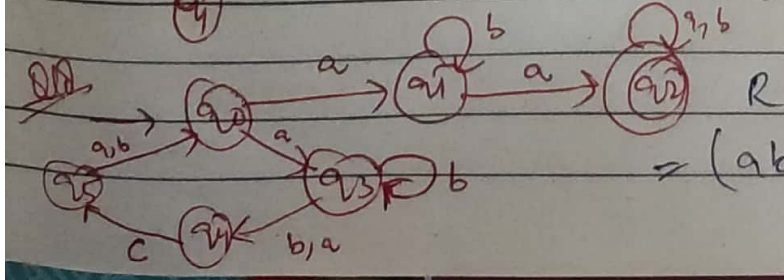
Q17.



$R.E =$

$(aa)^* a b^* a (a+b)^*$

Q18.

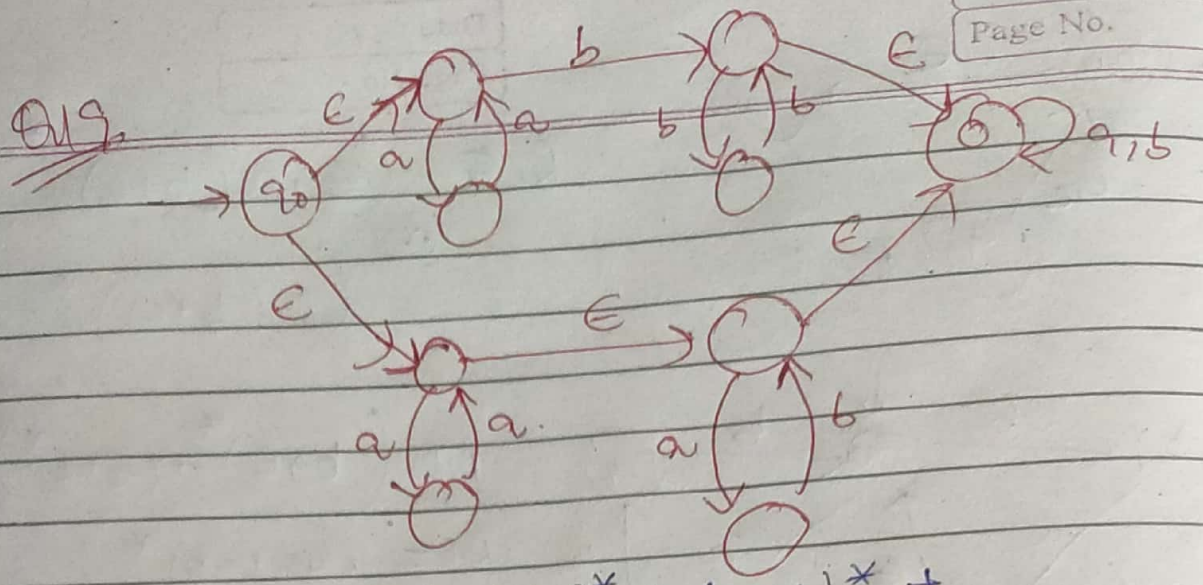


$R.E$

$= (ab^*(b+a)c(a+b))^* a b^* a (a+b)^*$.

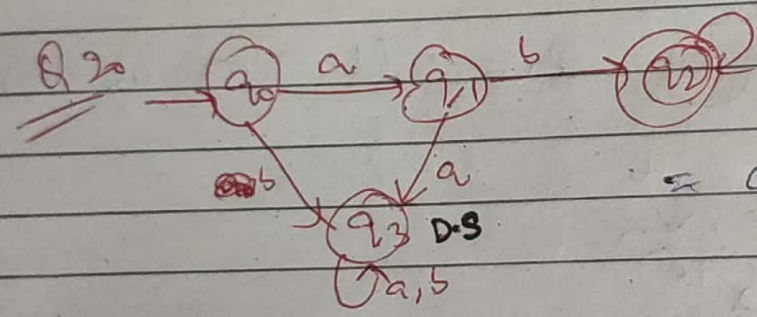
Date : / /
Page No.

Q19.



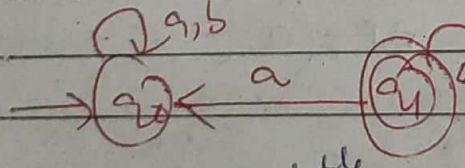
$RE = \epsilon(aa)^* b(bb)^* \epsilon (a+b)^* +$
$\epsilon(aa)^* \epsilon (ab) \epsilon (a+b)^*$
(or) $(aa)^* b(bb)^* (a+b)^* + (aa)^* (ab)^* (a+b)^*$.

Q20.



$= a.b(a+b)^*$

Q21.

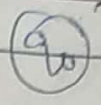


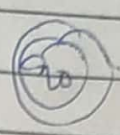
Non terminal state

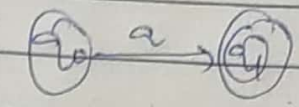
~~(a+b)~~ $RE = \phi$

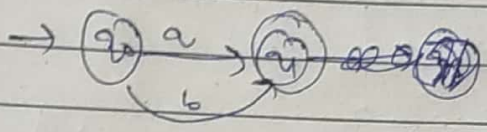
Date : / /
Page No.

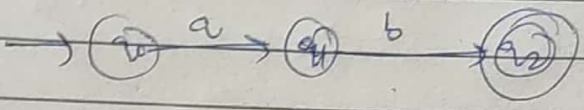
Conversion of R.E to F.A.

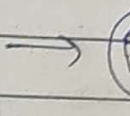
1) ϕ 

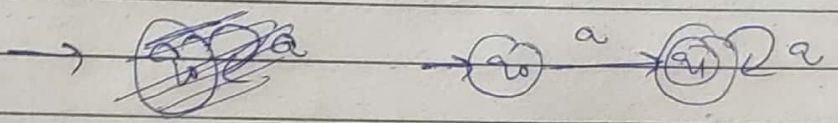
2) ϵ 

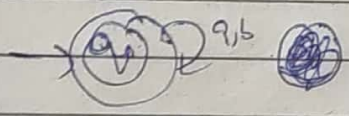
3) a 

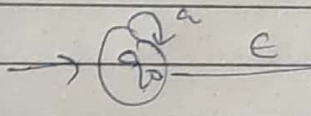
4) $a+b$ 

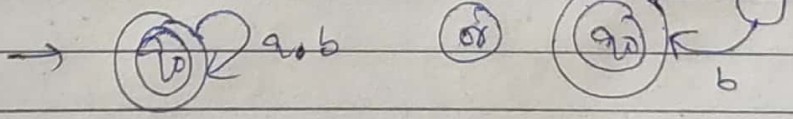
5) $a.b$ 

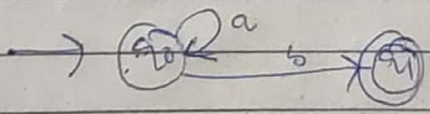
6) a^* 

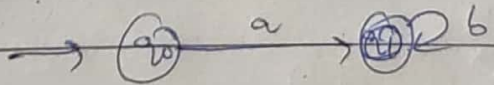
7) a^+ 

8) $(a+b)^*$ 

9) a^*b^* 

10) $(ab)^*$ 

11) a^*b 

12) ab^* 

13) a^*bc^*